N2NHU Labs

Book Thirty-Seven

N2NHU Open Source Ethernet Switch

Layer II Internet Bridge

Design and Theory of Operation

J P Ames

N2NHU Labs · Newburgh, New York

April 2026

N2NHU Open Source Ethernet Switch / Layer II Internet Bridge: Design and Theory of Operation

First Edition · April 2026

Copyright © 2026 J P Ames / N2NHU Labs

Newburgh, New York, United States

The reference implementation described in this book is released under the GNU General Public License version 3.

The text of this book may be reproduced for educational use with attribution.

Trademarks of third-party products mentioned in this book are the property of their respective owners.

For the engineers who still read source code

and for the students who will build what comes next.

Contents

Right-click the table below in Microsoft Word and choose Update Field to populate page numbers.

Preface

This is a book about a small program. The program is roughly seventeen hundred lines of Python that, taken together, implements an Ethernet switch in software, tunnels its switching plane across the public internet using ngrok as the underlay, and exposes the result as a flat broadcast domain to whatever hosts choose to attach. The program is called Brooklyn Bridge. It is one of the open source releases of N2NHU Labs and the subject of this, the thirty-seventh book in the N2NHU Labs series.

The book is short by the standards of network engineering literature and long by the standards of source-code commentary. It sits between these two genres deliberately. A reader who comes for the theory will find the underlay/overlay model, the mechanics of MAC learning, the rationale for length-prefixed framing, the trade-offs between TCP and UDP underlays, and the security properties that follow from each design choice. A reader who comes for the code will find structural walkthroughs of every major class with key listings reproduced inline. A reader who comes to evaluate the project against the broader ecosystem of overlay networking products will find a vendor-by-vendor feature matrix, written without partisanship, that situates Brooklyn Bridge among ZeroTier, Tailscale, n2n, Cisco SD-WAN, Fortinet Secure SD-WAN, and the major adjacent products.

Brooklyn Bridge was not designed to compete with commercial SD-WAN platforms. It was designed to solve a recurring problem at N2NHU Labs: customers and collaborators need their hosts to behave as if they were on the same Ethernet segment, regardless of which Internet Service Provider, which Network Address Translator, or which carrier-grade firewall sits between them. Layer-3 VPNs route IP packets between subnets; they do not preserve the broadcast and multicast semantics on which Layer-2 service discovery, industrial control protocols, and zero-configuration networking depend. The need for a small, readable, GPL-licensed Layer-2 bridge — one whose source could be modified, audited, and redeployed without committing to a commercial vendor — produced the program described in this book.

The program is small enough to fit in one's head. The book exists because this is no longer common. Modern infrastructure software is typically too large for a single engineer to read end-to-end, and modern networking products are typically opaque. Brooklyn Bridge is presented here as evidence that a working, observable Layer-2 overlay can be built and understood in a single weekend's reading. The text aims to make that reading easier.

How to read this book

Chapters 1 through 4 establish the problem space, the reasons for development, the architectural overview, and the position of Brooklyn Bridge among comparable products. They are conceptually self-contained and can be read by anyone with general networking literacy.

Chapters 5 through 8 develop the underlying theory: the underlay/overlay model, the wire protocol, the switching plane, the IP address management subsystem. These chapters are necessary for understanding the code that follows.

Chapters 9 through 12 are the structural code walkthrough. They reproduce key listings from the source and explain the surrounding logic. These chapters assume comfort with Python and basic socket programming. A reader without that background may skim them and proceed to Chapter 13.

Chapters 13 through 16 cover deployment, configuration, use cases, and pedagogical applications. They return the reader to the operator's perspective.

Chapter 17 surveys planned future directions and identifies non-goals. Chapter 18 concludes.

Three appendices follow: the wire protocol specification, the configuration file reference, and a glossary of terms.

On voice and convention

The book is written in formal third person. The reference implementation is referred to as Brooklyn Bridge or, when the immediate context is clear, as the bridge. References to specific source files, classes, and methods appear in monospaced type. Code listings are numbered by chapter and reproduce excerpts from the version 3.1.2 source as released under GPL v3 in April 2026. Listings are excerpts and not full files; the full source is available with the project distribution.

Trademarks of third-party products — including ZeroTier, Tailscale, n2n, ngrok, Cisco, Fortinet, VMware, OpenVPN, WireGuard, and others — appear without trademark symbols throughout. Each is the property of its respective owner.

1. Introduction

1.1 The subject of this book

Brooklyn Bridge is an open-source Ethernet switch implemented as a pair of Python programs. The server program runs on a host that has a virtual Ethernet adapter — a TAP device — installed; the client program runs on each remote host that wishes to be attached to the bridge. Together, the server and any number of clients constitute a single broadcast domain in which Ethernet frames are forwarded between participants according to standard learning-bridge semantics. The transport that carries frames between client and server is an ngrok TCP tunnel by default, though any reliable byte-oriented transport will serve.

The phrase "Layer II Internet bridge" in the book's title is precise. Brooklyn Bridge operates at the second layer of the OSI reference model, the data-link layer, where the unit of transmission is the Ethernet frame and the unit of addressing is the 48-bit Media Access Control address. The bridge does not route Internet Protocol packets, does not maintain a routing table, does not implement Network Address Translation, and does not impose any opinion on the IP-layer protocols that travel across it. Hosts connected through the bridge appear to one another as if they shared a single Ethernet segment.

The reference implementation — version 3.1.2 as of this writing — is approximately 1,750 lines of Python distributed across two files. It depends on the Python standard library and, on Microsoft Windows, on the OpenVPN TAP-Windows driver and the pywin32 module that wraps the Win32 file API. There are no other runtime dependencies. The implementation is small enough that its execution can be understood in detail by reading the source from end to end.

1.2 The intended audience

This book addresses three overlapping audiences. The first is the operator — the engineer who must deploy a Layer-2 overlay between geographically separated hosts and who needs to understand both how the bridge functions and where its operational limits lie. The second is the architect — the engineer comparing Brooklyn Bridge to commercial SD-WAN products, peer-to-peer mesh networks, and other open-source overlays, and who needs to understand where Brooklyn Bridge fits among them. The third is the student — the engineer or researcher who wishes to understand the principles of software-defined Ethernet switching by reading a complete working specimen.

The book is written under the assumption that all three readers benefit from the same material in different proportions. The operator-focused chapters explain configuration, deployment, troubleshooting, and operational behavior. The architectural chapters develop the theory of overlay networking and locate Brooklyn Bridge against competing approaches. The pedagogical chapters identify the points at which the implementation illustrates fundamental concepts in networking — MAC learning, broadcast containment, frame validation, length-prefix framing, lease-based address allocation — and propose exercises through which a student may explore these concepts on running code.

1.3 The name

The reference implementation is called Brooklyn Bridge after the Roebling-designed suspension structure that connects the boroughs of Brooklyn and Manhattan across the East River. The metaphor is exact in two senses. First, the bridge spans a separating medium — for the eastern bridge, the river; for the network bridge, the public internet — and joins two otherwise disjoint segments into a single traversable surface. Second, the bridge is fundamentally a passive structure: it does not transform traffic, does not impose tolls on individual frames, does not interpret the load it carries. It only carries.

The metaphor is imperfect in one respect worth noting. The eastern Brooklyn Bridge is a permanent structure; the network bridge described here is a process running on commodity hardware that may be restarted, redeployed, or dismantled as operational needs change. The name preserves the geometric image without claiming the permanence.

1.4 Outline of subsequent chapters

The remainder of this book proceeds as follows. Chapter 2 presents the reasons for development — the operational problems Brooklyn Bridge was designed to solve — and the design constraints that followed. Chapter 3 provides an architectural overview at a level of detail suitable for an engineer evaluating the project for deployment. Chapter 4 positions Brooklyn Bridge among comparable products through a vendor-by-vendor feature matrix.

Chapters 5 through 8 develop the theory of operation. Chapter 5 examines the underlay/overlay model that all overlay networks share. Chapter 6 specifies the TAP virtual network adapter and the wire protocol Brooklyn Bridge speaks across the underlay. Chapter 7 walks through the source-learning Ethernet switch at the heart of the implementation. Chapter 8 covers the IP address management subsystem and the session model.

Chapters 9 through 12 conduct the structural code walkthrough. Chapter 9 traces the server's main path from configuration loading through accept loop. Chapter 10 walks the per-client session class and its receive and transmit loops. Chapter 11 covers the client implementation, focusing on its symmetry with and divergences from the server. Chapter 12 examines the test harness — a 42-test integration suite that exercises the protocol layer end to end.

Chapters 13 through 16 return to operational concerns. Chapter 13 covers deployment of both server and client. Chapter 14 documents the configuration files in detail. Chapter 15 surveys use cases — IoT troubleshooting, zero-configuration deployment, industrial control system extensibility, and others. Chapter 16 develops the pedagogical applications: laboratory exercises through which the implementation may be used to teach Ethernet switching.

Chapter 17 identifies known limitations and planned future directions. Chapter 18 concludes.

2. Reasons for Development

This chapter situates Brooklyn Bridge in the operational context that produced it. The argument is that the contemporary internet has, by virtue of its ubiquitous deployment of Network Address Translation and its architectural commitment to client-server topologies, made Layer-2 reachability between arbitrary hosts unusually difficult — and that the difficulty falls hardest on operators of industrial control systems, Internet-of-Things deployments, and small-shop infrastructure where the burden of commercial SD-WAN tooling cannot be justified.

2.1 The Layer-2 reachability problem

A foundational property of Ethernet is the broadcast domain. Devices attached to the same Ethernet segment can address one another by Media Access Control address, can resolve IP addresses to MAC addresses by the Address Resolution Protocol, can announce services by multicast Domain Name System traffic, and can rely on link-local addressing in the absence of any centrally administered allocation scheme. These properties are foundational because every protocol layered above Ethernet — including the Internet Protocol itself — was designed under the assumption that the link layer provides them.

The historical trajectory of the internet has erased this assumption everywhere except within individual local networks. Wide-area connectivity is now invariably routed at Layer 3, with the result that hosts on different networks cannot exchange Layer-2 frames except through tunneling protocols specifically designed for the purpose. The protocols that depend on broadcast and multicast service — service discovery, link-local addressing, much of the industrial control system landscape — therefore stop functioning the moment the relevant hosts are placed on different IP networks. A protocol designed under the assumption of a flat broadcast domain breaks at the first router.

Brooklyn Bridge addresses this problem by reconstructing a flat broadcast domain across whatever wide-area transport is available. The transport need not preserve Ethernet semantics — it need only deliver bytes reliably — because the bridge encapsulates Ethernet frames within its own protocol and reproduces broadcast and multicast behavior at the encapsulation layer. From the perspective of hosts attached to the bridge, the wide-area network has been replaced by a single Ethernet switch.

2.2 The NAT and CGNAT obstacle

A second compounding factor is the prevalence of Network Address Translation. The exhaustion of the IPv4 address space, beginning in earnest in the 2010s, drove residential and small-business Internet Service Providers toward Carrier-Grade NAT, in which large numbers of customer connections share a single public IPv4 address. Devices behind a CGNAT cannot accept inbound connections directly; they can only initiate outbound connections, with the carrier's NAT rewriting source addresses on the wire. This property is not specific to CGNAT — every NAT, large or small, has it — but CGNAT removes the customer's

ability to circumvent the obstacle by configuring port forwarding on a router under the customer's own control.

Operators who must reach hosts behind such NATs cannot rely on direct IP connectivity. The customer must be persuaded to install software, the customer's gateway must be persuaded to permit outbound connections from that software, and the connection must be terminated at a remote endpoint with stable, publicly reachable addressing. This pattern — customer-initiated outbound tunnels to a centrally managed concentrator — has emerged as the dominant solution. Commercial SD-WAN platforms implement it, mesh VPN products implement it, and Brooklyn Bridge implements it.

The bridge's reliance on ngrok is a specific implementation of this pattern. ngrok establishes an outbound TCP tunnel from the operator's server host to a publicly addressed concentrator operated by ngrok Inc.; clients reach the server by connecting to the public hostname and port that ngrok publishes for the tunnel. The tunnel itself traverses NAT in the same manner as any outbound TCP connection. The choice of ngrok is convenient rather than essential — any reliable byte-oriented transport will serve in its place — and Chapter 13 documents alternative transports.

2.3 The triggering case

The decision to develop Brooklyn Bridge was precipitated by a specific operational case in November 2025. A customer of N2NHU Labs operated an industrial pump station whose programmable logic controller exposed both Modbus over TCP for normal operation and a vendor-specific Layer-2 multicast discovery protocol for configuration. The PLC was located behind a cellular gateway with carrier-grade NAT; the operator's diagnostic tooling, running on a workstation in the operator's office, used multicast discovery to identify PLCs on the local network before establishing a TCP session for configuration changes.

Existing remote access tooling solved a fraction of the problem. A Layer-3 VPN — WireGuard or OpenVPN in routed mode — could deliver IP packets between the office and the pump station, but multicast discovery did not cross the routed boundary, so the diagnostic tool could not identify the PLC. A reverse SSH tunnel could expose the Modbus port, but the operator would have to configure the Modbus connection by IP address rather than by the discovered identity, and many of the diagnostic tool's other features depended on the multicast announcements. Commercial SD-WAN products supported Layer-2 bridging but the licensing cost was disproportionate to a single-customer deployment, and the lead time to procure and configure the equipment was measured in weeks rather than the days the operational situation permitted.

A small Layer-2 bridge, built specifically to span ngrok and small enough to deploy on commodity Windows mini-PCs, would solve the immediate problem and, with appropriate generalization, would be useful in subsequent cases. The first version of Brooklyn Bridge was operational two weeks after the triggering case, with the customer's diagnostic tool functioning identically to its behavior on the local network. The implementation has since been generalized through several revisions, summarized in the version history

below, and now serves as the foundation for an SD-WAN-style overlay across multiple N2NHU Labs deployments.

2.4 Adjacent motivations

Several adjacent motivations sustained the project beyond its triggering case. The first was pedagogical: the implementation was small enough that it could serve as a worked example in instruction on software-defined networking, and the existing literature in this area — while voluminous on the theory side — contained few examples of complete, readable implementations at a comparable scale. The second was philosophical: the commercial SD-WAN landscape had, by 2025, become opaque. The mesh VPN products that had risen to prominence in the late 2010s — Tailscale, ZeroTier, and others — were either partially closed-source or had moved portions of their code to source-available licenses requiring commercial fees for non-personal deployment. A small, fully GPL-licensed reference implementation of a Layer-2 overlay served as a deliberate counter to this trend.

The third motivation was technical: the implementation effort itself surfaced design questions that, in the operator's judgment, were worth answering through code rather than only through architectural debate. The decision to use TCP rather than UDP as the underlay raised questions about head-of-line blocking and tunnel-over-tunnel performance. The decision to key IP address management on session identifiers rather than client labels raised questions about identity, spoofing, and graceful reconnection. The decision to implement broadcast rate limiting raised questions about the failure modes of Layer-2 networks under storm conditions. Each of these decisions is documented in the chapters that follow, with both the choice made and the alternatives considered.

2.5 Version history of the reference implementation

The reference implementation has progressed through several revisions, each of which addressed specific operational issues. The version 3.1.1 release introduced the session-key model that decoupled IP address management from client labels, eliminating an identity-spoofing pathway that had been present in earlier versions. The version 3.1.2 release, which is the version this book documents, added five corrections identified during a structured triage pass: conditional Win32 imports to permit non-Windows portability of the protocol layer; defensive runt-frame handling to prevent contamination of the MAC learning table; defensive multicast-detection helpers to prevent crashes on malformed inputs; explicit socket shutdown on graceful client disconnect to expedite server-side session cleanup; and runt-frame rejection on the client side to prevent invalid frames from crossing the underlay. Each of these corrections was accompanied by a regression test in the integration suite documented in Chapter 12.

3. Architectural Overview

This chapter describes the structure of Brooklyn Bridge at a level of abstraction suitable for an engineer evaluating the project for deployment. Implementation details are deferred to subsequent chapters; the focus here is on the principal components, their relationships, and the data flow that connects them.

3.1 The two-network model

Every overlay network is, in essence, a system that uses one network as the underlying transport for another. The terminology that has converged in the SD-WAN and mesh VPN communities is that the carrying network is the underlay and the carried network is the overlay. The same vocabulary is used for VXLAN within data centers, for GRE-based tunnels in service provider networks, and for the various peer-to-peer mesh products. Brooklyn Bridge follows this convention exactly.

The underlay for Brooklyn Bridge, in its default configuration, is a TCP connection through an ngrok tunnel. The choice of ngrok is operational rather than architectural — ngrok is a convenient way to obtain stable, publicly reachable connectivity from a host behind a NAT — and any reliable byte-oriented underlay will serve. Plain TCP between two publicly reachable hosts will serve. An SSH local forward will serve. A Cloudflare Tunnel will serve. The bridge does not interpret the underlay; it requires only that bytes written at one end be delivered, in order, at the other end.

The overlay is a flat Ethernet broadcast domain. Hosts attached to the bridge — whether through the server's local TAP device or through any of the connected clients — exchange Ethernet frames as if they shared a single physical switch. ARP requests broadcast across all attached hosts. Multicast announcements reach all subscribers. Unicast traffic is forwarded according to the standard learning-bridge algorithm. The semantics of the overlay are precisely those of a small unmanaged Ethernet switch.

3.2 Principal components

The reference implementation is decomposed into a small set of components, each of which corresponds to a distinct concern. The components are:

- The wire framing layer. This component is responsible for delineating message boundaries on the underlying TCP stream. Each message consists of a five-byte header — one byte indicating the message type and four bytes indicating the payload length in big-endian order — followed by the payload of the indicated length. The framing layer is shared between server and client and is implemented in the FramedSocket class on each side.
- The TAP device wrapper. This component provides a uniform interface to the operating system's virtual Ethernet adapter. On Microsoft Windows, the wrapper uses the OpenVPN TAP-Windows driver via the Win32 file API exposed through pywin32. The wrapper is implemented in the TapDevice class and is one of the few pieces of platform-dependent code in the project.

- The switch plane. The switch plane maintains the forwarding database — the table that records, for each known MAC address, the port through which frames addressed to that MAC should be forwarded. The switch plane implements the standard learning-bridge algorithm: it records the source MAC of every received frame, looks up destination MACs against the table, floods frames with unknown destinations, and ages out entries that have not been observed within a configured interval. The switch plane is implemented in the SwitchPlane class.
- The IP address management subsystem. This component allocates IP addresses from a configured pool to clients that complete the handshake. Allocations are persisted to a SQLite database to survive server restarts. The subsystem is implemented in the SQLiteIPAM class.
- The token-bucket rate limiter. This component constrains the rate at which broadcast and multicast frames can be flooded to connected clients. Two buckets are maintained: one global bucket for the entire bridge, one per-client bucket for each connected session. The rate limiter is implemented in the TokenBucket class.
- The per-client session. Each connected client is represented on the server by an instance of the ClientSession class, which owns the underlying socket, the framed message queue, and the receive and transmit threads dedicated to that client. The session class is the principal locus of per-client state.
- The server orchestrator. The VPNServerV311 class instantiates the components above and coordinates their interaction. It owns the listening socket, the accept loop, the housekeeping thread that prunes dead sessions, and the threads that connect the local TAP device to the switch plane.
- The client orchestrator. The VPNClientV311 class is the symmetric counterpart on the client side. It owns the underlying socket, the local TAP device, the framing layer, and the threads that connect the two.

3.3 Data flow

The data flow within Brooklyn Bridge is best understood by following a single Ethernet frame from origin to destination. Suppose host A, attached to the bridge through client A, transmits an Ethernet frame addressed to host B, attached through client B. The frame proceeds as follows.

The frame originates from host A's operating system, which writes it to the virtual TAP device. Client A's TAP receive loop reads the frame, validates its size, and pushes it onto the queue feeding the network transmit loop. The transmit loop wraps the frame in a DATA_ETHERNET message and writes the wrapped bytes to the underlay TCP socket.

The wrapped message traverses the underlay and arrives at the server. The server's per-client receive loop reads the message, recovers the original Ethernet frame, and presents it to the switch plane. The switch plane records the frame's source MAC against client A's session, looks up the destination MAC, and either forwards the frame to a specific client (if the destination is known) or floods it to all other attached ports (if the destination is unknown or is the broadcast address).

Suppose the destination MAC is known and corresponds to client B. The switch plane enqueues the frame on client B's transmit queue. Client B's per-client transmit loop reads from the queue, wraps the frame in a DATA_ETHERNET message, and writes the wrapped bytes to client B's underlay socket. The wrapped message traverses the underlay and arrives at client B.

Client B's network receive loop reads the message, recovers the Ethernet frame, and pushes it onto the queue feeding the local TAP transmit loop. The local TAP transmit loop writes the frame to client B's TAP device. Host B's operating system observes the frame as if it had been received on a physical Ethernet adapter and processes it accordingly.

The complete journey involves two TAP traversals, two underlay traversals, and one switch plane lookup. The latency is dominated by the underlay round-trip; the bridge itself contributes only the time required to copy the frame between queues and to perform the dictionary lookup in the switch plane. On modern hardware, the bridge's contribution is on the order of tens of microseconds; underlay latency typically dominates by three orders of magnitude or more.

3.4 Threading model

Brooklyn Bridge is multithreaded. Each long-running activity is implemented as a Python thread, with queues providing the inter-thread communication. The server runs the following threads simultaneously: one accept thread monitoring the listening socket; one TAP receive thread reading frames from the local TAP; one TAP transmit thread writing frames to the local TAP; one housekeeping thread pruning dead sessions and aging the MAC table; and, for each connected client, one receive thread and one transmit thread. The client runs a smaller set: one TAP receive thread, one TAP transmit thread, one network receive thread, and one network transmit thread.

The threading model is straightforward and adequate for the bridge's operational scale. With the default configuration, one server can host approximately fifty concurrent clients on commodity hardware before the thread-management overhead becomes significant. Higher counts require either an asynchronous I/O rewrite or a different concurrency model; the reasons to prefer the threading model at the current scale are documented in Chapter 17.

3.5 The TCP underlay choice

Brooklyn Bridge uses TCP rather than UDP as its underlay. This choice merits explicit justification because the conventional wisdom in VPN design favors UDP, on the grounds that TCP-over-TCP exhibits a pathological interaction in which both the inner and outer TCP layers retransmit on packet loss, and the two retransmissions interfere with one another to produce throughput collapse far worse than the underlying loss rate would suggest.

The conventional wisdom is correct as a generalization, but it does not strictly prohibit TCP underlays in all cases. The pathological interaction is most acute when the inner traffic is itself bandwidth-saturating TCP,

when the underlying loss rate is non-trivial, and when the operator has bandwidth-throughput as a primary success criterion. Brooklyn Bridge's primary use cases — IoT troubleshooting, occasional administrative access to remote equipment, small-scale industrial control system extensibility — are not bandwidth-saturating in this sense. The operational reality is closer to bursty, low-volume management traffic over a tunnel that, regardless of throughput characteristics, must traverse arbitrary NATs without operator-controlled port forwarding.

The choice of TCP also has a practical motivation: ngrok's free service tier offers TCP tunnels but requires a paid subscription for UDP. The decision to use TCP allowed the bridge to be deployed without imposing a subscription cost on operators. For deployments in which throughput dominates and a UDP underlay is operationally feasible, the bridge's protocol layer is independent of the underlying transport and would function over UDP with minor modifications; this is identified as a possible direction for future work in Chapter 17.

> **Design choice** *The throughput-collapse pathology of TCP-over-TCP is real but its operational impact is often overstated. The pathology assumes a loss-prone underlay and a saturating inner workload. Brooklyn Bridge typically runs over a well-conditioned underlay — ngrok's TCP tunnels generally operate at near-zero loss rates within a single hosting region — and carries non-saturating workloads. Operators considering Brooklyn Bridge for high-throughput scenarios should benchmark in their specific environment rather than reject the design on theoretical grounds alone.*

4. Position Among Comparable Products

This chapter situates Brooklyn Bridge among the products with which it competes for operator attention: commercial SD-WAN platforms, peer-to-peer mesh VPNs, and other open-source overlays. The intent is not to argue that Brooklyn Bridge is uniformly superior — it is not — but to identify the operational scenarios in which each product is the appropriate choice and to characterize Brooklyn Bridge's particular niche within that landscape.

4.1 The relevant comparators

Six products are surveyed in detail. Three are commercial: Cisco SD-WAN (formerly Viptela), VMware SD-WAN (formerly VeloCloud), and Fortinet Secure SD-WAN. These represent the enterprise SD-WAN market and are the primary competitors when the deployment scale justifies a vendor relationship. Three are open or quasi-open: Tailscale (built on WireGuard, with a managed control plane), ZeroTier (custom protocol, partially source-available since version 1.16), and n2n 3.0 from ntop (an established peer-to-peer Layer-2 VPN with a long lineage). Several smaller projects — Netbird, Netmaker, Headscale, Innernet — share the conceptual space but are omitted from the detailed matrix to keep the comparison tractable.

The features against which each product is evaluated were chosen to illuminate operational differences rather than to favor any particular product. They include the layer at which the overlay operates (Layer 2 vs. Layer 3); the underlay transport family (UDP, TCP, or both); the licensing model and self-hosting feasibility; the supported broadcast and multicast semantics; the operational dependency on a vendor-controlled control plane; the deployment complexity for a single small site; and the bandwidth characteristics achievable in typical deployments.

4.2 The feature matrix

The matrix below summarizes the principal differences. A reading of the matrix should be undertaken with awareness of three caveats. First, the values reflect default configurations; many products can be configured outside their defaults, and several of the entries change with non-default deployment choices. Second, the open-source products (Tailscale, ZeroTier, n2n) have changed over time, sometimes in operationally significant ways — ZeroTier's 1.16 license change being the most recent prominent example — and the matrix reflects the state of each product as of early 2026. Third, the matrix entries for commercial products reflect the configurations most commonly encountered in mid-market deployments; large-enterprise variants of each product offer additional capabilities not summarized here.

Table 4-1. Comparison of overlay networking products.

Feature	Brooklyn Bridge	Tailscale	ZeroTier	n2n 3.0	Cisco SD-WAN	Fortinet SD-WAN	VMware SD-WAN
OSI layer	Layer 2	Layer 3	Layer 2	Layer 2	Layer 3	Layer 3	Layer 3

Feature	Brooklyn Bridge	Tailscale	ZeroTier	n2n 3.0	Cisco SD-WAN	Fortinet SD-WAN	VMware SD-WAN
Underlay transport	TCP (default)	UDP, TCP fallback	UDP	UDP, TCP optional	UDP/IPsec	UDP/IPsec	UDP/IPsec
License	GPL v3	Client open / server proprietary	MPL 2.0 client; controller source-available since 1.16	GPL v3	Commercial	Commercial	Commercial
Self-hostable control plane	Yes (no separate control plane)	Via Headscale	Yes (with caveats since 1.16)	Yes	No	No	No
Broadcast/ multicast preserved	Yes	No	Yes	Yes	No (Layer 3)	No (Layer 3)	No (Layer 3)
Encryption (default)	Underlay-provided (ngrok TLS)	WireGuard (ChaCha20-Poly1305)	Salsa20/ Poly1305 + Curve25519	AES (configurable)	IPsec	IPsec	IPsec
Mutual authentication (default)	No (in v3.1.2)	Yes	Yes	Pre-shared key	Certificate-based	Certificate-based	Certificate-based
NAT traversal	Outbound TCP via ngrok	Hole punching + DERP relay	Hole punching + root relay	Hole punching + supernode relay	Tunnel via vBond	Tunnel via FortiManager	Tunnel via VCO
Topology	Hub-and-spoke	Mesh + DERP	Mesh + root	Mesh + supernode	Hub-and-spoke / mesh	Hub-and-spoke / mesh	Hub-and-spoke / mesh
Centralized orchestrator	None	Required	Required	None (CLI driven)	vManage	FortiManager	VCO
Free tier (deployment scale)	Unlimited (open source)	100 devices	25 devices	Unlimited (open source)	None	None	None
Zero-touch provisioning	Pre-image client INI	SSO sign-in	Network ID + approval	Community + key	PnP server flow	FortiZTP cloud	Activation link
Bandwidth profile	Modest (TCP-on-TCP)	High (kernel WireGuard)	Moderate	Moderate	High	High	High
Source size (lines)	~1,750 (Python)	Hundreds of thousands (Go)	Tens of thousands (C++)	~30,000 (C)	Proprietary	Proprietary	Proprietary
Operating cost (small site)	Free (or ngrok	Free at small scale	Free at small scale	Free	Thousands of dollars	Thousands of dollars	Thousands of dollars

Feature	Brooklyn Bridge	Tailscale	ZeroTier	n2n 3.0	Cisco SD-WAN	Fortinet SD-WAN	VMware SD-WAN
	subscriptio n)				per year	per year	per year
Audit feasibility (single engineer)	Yes (whole codebase)	Practical for client only	Practical for client only	Practical (large but readable)	Not feasible	Not feasible	Not feasible
Implementation maturity	Early (2025-2026)	Production at scale	Production at scale	Production for ~15 years	Production at scale	Production at scale	Production at scale
Native client platforms	Windows (TAP); protocol layer portable	Win/Mac/ Linux/ iOS/ Android	Win/Mac/ Linux/BSD/ iOS/ Android	Win/ Linux/Mac	Hardware appliance / VM	Hardware appliance / VM	Hardware appliance / VM

4.3 Reading the matrix

Several patterns emerge from the matrix that warrant explicit commentary.

First, the Layer-2 column. Of the seven products surveyed, only Brooklyn Bridge, ZeroTier, and n2n preserve Ethernet broadcast and multicast semantics by virtue of operating at the data-link layer. Tailscale and the three commercial SD-WAN platforms all operate at the network layer and route IP packets between subnets. For operators whose use case requires multicast service discovery, link-local addressing, or industrial control protocols that depend on broadcast announcement, the choice is among the three Layer-2 products. Among those three, n2n is the most mature and ZeroTier the most ergonomic; Brooklyn Bridge offers the smallest readable codebase and the strictest open-source license.

Second, the licensing column. The commercial SD-WAN products are obviously closed. The mesh VPNs are mixed: Tailscale's server is proprietary (with Headscale providing an open-source alternative coordination plane), and ZeroTier's controller code moved to a source-available license in version 1.16 of March 2026, with the consequence that operators wishing to self-host a controller must either accept the source-available terms or remain on a 1.14-or-earlier release. Among unrestrictedly open-source options, only Brooklyn Bridge and n2n remain on conventional GPL terms.

Third, the encryption row. Brooklyn Bridge in version 3.1.2 does not implement application-layer encryption; it relies on the underlay (typically ngrok's TLS) to provide confidentiality. This is a practical choice for the project's current development stage but it is a real limitation: an operator who cannot trust the underlay must either not deploy Brooklyn Bridge or wait for the version 13 release in which TLS with mutual authentication is planned. The other surveyed products all provide application-layer encryption by default. Operators who require encryption end-to-end at the bridge layer should select one of those products.

Fourth, the bandwidth profile row. Brooklyn Bridge's TCP-on-TCP underlay caps its throughput well below what a UDP underlay would permit. For the bandwidth-modest use cases the bridge was designed to serve, this is acceptable. For high-throughput scenarios — site-to-site backups, media streaming, large file transfers — a UDP-based product (Tailscale or one of the IPsec-based commercial platforms) will substantially outperform the bridge.

Fifth, the audit feasibility row. This row is unusual in a feature matrix and deserves explicit defense. The ability of a single engineer to read and understand the entire implementation of a system has operational consequences for security review, debugging in production, and modification under unforeseen circumstances. For commercial SD-WAN products, audit by a single engineer is not feasible; the source is unavailable and the codebases are too large in any case. For Tailscale and ZeroTier, audit of the client is practical but the server-side coordination plane is either closed or large enough to require team-scale effort. For n2n, audit is practical though the codebase is sufficiently large to require sustained attention. For Brooklyn Bridge, the entire system fits within one weekend's reading. This is not a substitute for security review but it is a different and valuable property; it permits an operator to know precisely what the bridge does.

4.4 Where Brooklyn Bridge is the appropriate choice

Brooklyn Bridge is the appropriate choice for an operator whose use case has the following properties simultaneously: a requirement for Layer-2 semantics (broadcast, multicast, ARP); a low-budget environment in which commercial SD-WAN licensing cannot be justified; a small operational scale (typically below fifty concurrent connected hosts); a willingness to operate in a trusted-underlay model in which the bridge itself does not enforce mutual authentication; and a preference for source code that can be modified under GPL terms. Many small-shop IoT troubleshooting deployments, hobbyist homelab interconnects, and educational laboratory deployments meet these criteria.

4.5 Where another product is the appropriate choice

Brooklyn Bridge is not the appropriate choice when any of the following conditions hold. First, when high bandwidth is a primary requirement: a UDP-based product such as Tailscale will substantially outperform the bridge. Second, when end-to-end mutual authentication is required by policy or compliance: the bridge in version 3.1.2 does not provide it, and operators with this requirement must select a product that does. Third, when the operational scale exceeds approximately fifty concurrent clients per server: the bridge's threading model becomes inefficient at higher counts, and an asynchronous I/O reimplementation has not yet been completed. Fourth, when the operator requires a vendor support contract: no such contract is available for the bridge.

In each of these cases, the alternative product is identified explicitly in the matrix above. Tailscale is the appropriate alternative for high-bandwidth, Layer-3, polished-tooling deployments. ZeroTier is the alternative for Layer-2 deployments with mature ecosystem integration. n2n is the alternative for Layer-2

deployments with established production track record. Cisco SD-WAN, Fortinet Secure SD-WAN, and VMware SD-WAN are the alternatives for enterprise deployments where vendor support, hardware appliances, and centralized policy management are operational requirements.

5. The Underlay/Overlay Model

This chapter develops the conceptual foundation that all overlay networks share. The vocabulary of underlay and overlay is reviewed in Section 5.1; the encapsulation operation that connects them is examined in Section 5.2; the trade-offs between Layer-2 and Layer-3 overlays are analyzed in Section 5.3; and the specific underlay choices available to Brooklyn Bridge operators are surveyed in Section 5.4.

5.1 Underlay and overlay defined

An overlay network is a logical network constructed atop a different physical or logical network. The carrying network is termed the underlay; the carried network is the overlay. The terminology is sufficiently general that it admits many examples: the original Internet itself was, in this sense, an overlay above the AT&T long-distance telephone network in its earliest deployments; a virtual local area network within an enterprise switching fabric is an overlay above the underlying Ethernet topology; a peer-to-peer file-sharing network is an overlay above the public internet. The pattern recurs whenever a system requires properties — addressing, routing, broadcast semantics, security policy — that the carrying network does not natively provide.

In contemporary practice, the term "overlay network" most often refers to a virtual network that uses the public internet as its underlay and reproduces the semantics of a private local area network at the overlay. This is the sense in which Brooklyn Bridge is an overlay. The underlay is whatever transport delivers bytes between Brooklyn Bridge's server and its clients — by default, an ngrok TCP tunnel — and the overlay is a flat Ethernet broadcast domain in which connected hosts exchange Ethernet frames as if attached to a single switch.

The cleanest formulation of the relationship is that the underlay knows nothing of the overlay, and the overlay knows nothing of the underlay. The underlay sees only opaque byte streams flowing between client and server; it does not parse the frames within those streams nor concern itself with their addressing semantics. The hosts attached to the overlay see only Ethernet frames flowing between Ethernet adapters; they do not know that the frames are being encapsulated and tunneled. Each layer is intentionally ignorant of the other; this ignorance is what permits the system to be reasoned about in pieces.

5.2 The encapsulation operation

An overlay network is connected to its underlay by an encapsulation operation. Frames or packets generated at the overlay are wrapped in an outer envelope that is meaningful to the underlay; the envelope is removed at the receiving end, and the original frame or packet is delivered into the overlay. The envelope is the encapsulation header. Different overlay technologies use different encapsulations: VXLAN uses a UDP envelope with a 24-bit virtual network identifier; GRE uses a generic Layer-3 envelope

with a protocol type field; IPsec in tunnel mode uses an outer IP header with the original packet as payload.

Brooklyn Bridge's encapsulation is straightforward. Each Ethernet frame to be tunneled is preceded by a five-byte header consisting of one byte indicating the message type (DATA_ETHERNET in this case) and four bytes indicating the payload length in big-endian unsigned integer format. The encapsulated message is written to the underlying TCP socket. At the receiving end, the five-byte header is read first, the payload length is decoded, and the indicated number of bytes is read as the original Ethernet frame. The frame is then delivered into the receiving end's overlay context — written to the TAP device on a client, presented to the switch plane on the server.

The encapsulation overhead is therefore five bytes per frame. For typical Ethernet frames carrying useful payload — say, 1500 bytes of IP data — the overhead is approximately 0.3 percent. The choice of a length-prefixed framing rather than a delimiter-based or fixed-size framing is dictated by the requirement to handle arbitrary frame sizes (broadcast and multicast frames are typically small; bulk data frames may be at the link MTU) without imposing a costly delimiter scan or wasting bytes on padding.

Listing *5-1. The five-byte frame header structure as defined in vpn_server.py.*

```
MSG_DATA_ETHERNET = 0x01
MSG_HELLO         = 0x02
MSG_CONFIG        = 0x03
MSG_HEARTBEAT     = 0x04
MSG_STATS         = 0x05
MSG_ERROR         = 0x06

FRAME_HDR = struct.Struct("!BI")  # Type (1 byte) + Length (4 bytes BE)
```

5.3 Layer-2 versus Layer-3 overlays

A consequential design decision in any overlay network is the layer at which the overlay operates. Layer-2 overlays present a flat broadcast domain at the encapsulation boundary and forward frames according to MAC-address learning. Layer-3 overlays present a routed network and forward IP packets according to destination IP address. Each choice has consequences that propagate through the rest of the system.

A Layer-2 overlay preserves Ethernet semantics. Broadcasts cross the overlay; multicasts cross the overlay; ARP and Neighbor Discovery function in their conventional form; link-local addressing functions; service discovery protocols such as multicast DNS function. The cost of preserving these semantics is that broadcast and multicast traffic is replicated to every connected client, which limits the number of clients that can be supported and creates a vulnerability to broadcast storms. The mitigations — rate limiting, broadcast suppression, MAC table aging — must be implemented carefully because Layer-2 networks, in their natural state, lack the loop-prevention and policy mechanisms that exist at higher layers.

A Layer-3 overlay does not preserve broadcast semantics. Packets that would have flooded the broadcast domain on a Layer-2 network simply do not cross the boundary on a Layer-3 network; they are restricted

to whichever segment they originated on. This eliminates the broadcast-storm vulnerability and permits much larger deployment scales — Tailscale routinely supports networks with thousands of nodes — but breaks every protocol that depends on the broadcast domain. Operators of Layer-3 overlays must accept that mDNS service discovery will not work between segments, that link-local addressing will not work between segments, and that industrial control protocols requiring multicast announcement will not function across the overlay.

The decision is therefore use-case dependent. Operators whose protocols require Layer-2 semantics must accept the operational constraints of a Layer-2 overlay. Operators whose protocols are content with Layer-3 routing benefit from the simpler scaling characteristics of a Layer-3 overlay. Brooklyn Bridge serves the former category; products such as Tailscale serve the latter. The two categories are not in competition for the same operators.

> **Design choice** *It is sometimes proposed that a Layer-3 overlay can be retrofitted to handle Layer-2 semantics by, for example, proxying mDNS announcements at the overlay boundary or implementing a virtual ARP responder. Such proxies work for specific protocols but invariably break for the next protocol the operator's environment uses. A Layer-2 overlay handles all such protocols uniformly because it preserves Ethernet semantics by construction; this is its principal advantage.*

5.4 The underlay choice

Brooklyn Bridge's underlay is, as previously noted, an ngrok TCP tunnel by default. The choice is operationally convenient but architecturally arbitrary: the bridge's protocol layer requires only that the underlay deliver bytes reliably, in order, between the two endpoints. Several alternative underlays merit explicit mention.

Direct TCP between two publicly reachable hosts is the simplest underlay. It eliminates the dependency on ngrok and the latency overhead of routing through ngrok's edge servers. It requires that one of the two endpoints have a publicly reachable IP address, which is typically the operator's server. The client's address can be private. This underlay is appropriate when the operator controls a publicly addressable host and is comfortable opening an inbound TCP port on that host.

An SSH local forward through an intermediate jump host can serve as the underlay. The jump host need not run any custom software; it requires only an SSH server. The bridge's server runs on the jump host or on a host reachable from the jump host; the client tunnels its TCP connection through SSH. This underlay is appropriate when the operator already has SSH infrastructure and wishes to avoid additional complexity.

A Cloudflare Tunnel can serve as the underlay. Cloudflare's free tier offers stable inbound tunnels in a manner architecturally similar to ngrok's. The bridge's server runs on the operator's host with a Cloudflare Tunnel client; clients connect to the public hostname Cloudflare assigns. This underlay is appropriate when the operator already uses Cloudflare or wishes to avoid ngrok's session limits.

A WireGuard or OpenVPN tunnel can serve as the underlay. This produces the somewhat unusual configuration in which a Layer-2 overlay runs on top of a Layer-3 overlay; the operational rationale is that the Layer-3 overlay provides authenticated, encrypted transport to a known endpoint, and the Layer-2 overlay provides flat broadcast semantics atop that. The combination is heavyweight but provides defense-in-depth.

None of these alternatives has been extensively tested with Brooklyn Bridge in production at the time of writing. The default ngrok configuration has the most operational experience behind it. Operators selecting an alternative underlay should expect to invest some debugging effort in the integration.

6. The TAP Adapter and Wire Protocol

This chapter examines the two principal interfaces of Brooklyn Bridge: the TAP virtual network adapter that connects the bridge to the operating system's networking stack, and the wire protocol that carries encapsulated frames across the underlay. The TAP adapter is described in Section 6.1; the wire protocol's framing layer is in Section 6.2; the message types and their payloads are documented in Section 6.3.

6.1 The TAP virtual network adapter

A TAP device is a virtual Ethernet adapter implemented in software. From the operating system's perspective, it appears as a network interface card with a unique MAC address, can be assigned an IP address, can be configured to participate in routing, and can transmit and receive Ethernet frames. From the perspective of a userspace program with appropriate privileges, the TAP device exposes a file-like interface: writes to the file inject Ethernet frames into the operating system's networking stack as if they had been received from a wire; reads from the file return Ethernet frames that the stack has transmitted as if they were being delivered onto a wire.

The TAP device is the hinge that makes overlay networking practical at the data-link layer. Without it, a userspace program wishing to inject Ethernet frames into the local stack would have to install a custom kernel driver or operate in a privileged kernel mode. The TAP device provides this capability through a generic interface, freeing the userspace program to concern itself only with the data flowing through the adapter.

On Microsoft Windows, Brooklyn Bridge uses the OpenVPN TAP-Windows driver, which has been maintained by the OpenVPN community since the early 2000s and is the most widely deployed TAP driver on the platform. The driver exposes the TAP device through the standard Win32 file API; the program opens the device using CreateFile, transmits frames using WriteFile, and receives frames using ReadFile. The pywin32 library provides Python bindings to these calls. The TapDevice class in vpn_server.py and vpn_client.py wraps these primitives in a uniform interface.

On Linux, the equivalent functionality is provided by the /dev/net/tun device, which exposes TAP and TUN interfaces through standard file I/O. The Brooklyn Bridge codebase does not currently include a Linux TAP wrapper, but the protocol layer above the TAP is platform-independent and a Linux client could be implemented in approximately one hundred lines of additional code. This is identified as a target for future work in Chapter 17.

> **Note** *In version 3.1.2 of the reference implementation, the TAP wrapper is the only platform-dependent component. The Win32 imports are now conditional, permitting the protocol layer (and therefore the test harness) to load and execute on any platform with Python 3.10 or later. A reader on a non-Windows host can read, run, and modify the bulk of the codebase; only the actual TAP I/O requires Windows.*

6.2 The framing layer

The framing layer, implemented in the FramedSocket class on both server and client sides, is responsible for delineating message boundaries on the underlying TCP stream. This responsibility is necessary because TCP is a stream-oriented transport: the bytes written at one end arrive in the same order at the other, but TCP imposes no notion of message boundaries. A program that writes 100 bytes followed by 200 bytes has no guarantee that the receiver will observe two distinct reads of those sizes; the receiver may observe a single read of 300 bytes, or three reads of 100 bytes each, or any other partition of the 300 bytes into a sequence of reads.

The framing layer addresses this by prepending each message with a fixed-size header that specifies the message's length. The receiver reads the header first, decodes the length, and then reads exactly that many bytes as the message payload. The pattern is termed length-prefixed framing and is the standard solution to message delineation over a stream-oriented transport. Brooklyn Bridge's specific choice — five bytes of header consisting of one byte of message type and four bytes of length — is conventional for the problem and admits messages of up to four gigabytes (which is far larger than will ever be transmitted in practice; a guard limit of 256 kilobytes is enforced as a safety measure).

Listing 6-1 shows the receive side of the framing layer. The function reads exactly five bytes for the header, decodes the type and length, validates the length against the configured maximum, and then reads exactly that many bytes for the payload. Each read may require multiple underlying socket recv() calls because TCP is permitted to return fewer bytes than requested; the helper _recv_exact loops until either the requested count is satisfied or the connection closes.

Listing *6-1. The receive side of the framing layer (vpn_server.py).*

```
def recv_msg(self) -> Optional[Tuple[int, bytes]]:
    hdr = self._recv_exact(FRAME_HDR.size)
    if hdr is None:
        return None
    t, length = FRAME_HDR.unpack(hdr)
    if length > self.max_frame_bytes:
        raise ValueError(f"frame too large: {length} > {self.max_frame_bytes}")
    if length == 0:
        return (t, b"")
    payload = self._recv_exact(length)
    if payload is None:
        return None
    return (t, payload)
```

6.3 Message types

Brooklyn Bridge defines six message types, each of which is identified by a single byte in the framing header. The types and their semantics are summarized below.

DATA_ETHERNET (0x01)

Carries an encapsulated Ethernet frame. The payload is the raw frame as it would appear on the wire: destination MAC, source MAC, EtherType, and frame payload. The minimum valid payload size is fourteen bytes (the Ethernet header alone); the maximum is constrained by the configured ethernet_frame_max parameter, which defaults to 1600 bytes. Frames smaller than fourteen bytes are termed runts and are dropped; frames larger than the maximum are dropped with a counter increment.

HELLO (0x02)

Sent from client to server immediately after connection establishment. The payload is a JSON object containing client_id (an operator-chosen label), version (the client's protocol version string, currently "3.1.1"), and platform (an opaque platform identifier). The HELLO message is the trigger for the server to begin the handshake; the server allocates a session, draws an IP address from the IPAM, and responds with a CONFIG message.

CONFIG (0x03)

Sent from server to client in response to HELLO. The payload is a JSON object containing assigned_ip (the IP address the server has allocated to this client), subnet_mask, gateway, dns, mtu, and session_key (the server-assigned ephemeral session identifier). The client extracts these values and, depending on its apply_ip_config setting, may apply them to the local TAP device via netsh. The session_key is retained by the client as an opaque identifier.

HEARTBEAT (0x04)

Sent periodically (default every five seconds) from client to server. The payload is empty. The heartbeat serves as a liveness signal; the server records the time of the most recent message from each client and declares clients dead if no message has been received within the configured client_timeout_seconds (default thirty seconds). The heartbeat is not strictly necessary for correctness — any message from the client refreshes the liveness timestamp — but it ensures that an idle client which is generating no Ethernet traffic is not misclassified as dead.

STATS (0x05)

Reserved for transmission of statistics from server to client or client to server. The payload is a JSON object whose schema is implementation-defined. STATS is not currently transmitted in version 3.1.2; the message type is reserved for future use.

ERROR (0x06)

Sent from server to client to communicate a fatal error condition. The payload is a UTF-8-encoded human-readable error message. ERROR is sent in response to handshake failures and is followed immediately by connection termination. The client, on receiving ERROR, logs the message and either reconnects (if the error is transient) or terminates (if the error is permanent).

7. The Switching Plane

This chapter examines the source-learning Ethernet switch at the heart of Brooklyn Bridge. Section 7.1 reviews the switching algorithm in its abstract form. Section 7.2 walks through the implementation. Section 7.3 covers MAC aging. Section 7.4 examines broadcast and multicast handling and the rate-limiting that protects the switch against storm conditions.

7.1 The source-learning algorithm

The behavior of an Ethernet switch is governed by a remarkably simple algorithm. When a frame arrives on a port, the switch performs three operations: it records the frame's source MAC address against the port on which the frame arrived; it consults its forwarding table to determine the port for the frame's destination MAC address; and it forwards the frame to that port if the destination is known, or floods it to every port other than the ingress if the destination is unknown.

The algorithm has the property that, after a brief transient at startup during which the switch's forwarding table is empty, the switch converges to a state in which every active host is known and traffic is forwarded only to the appropriate port. Broadcast frames — those addressed to FF:FF:FF:FF:FF:FF — are flooded by definition, as are multicast frames whose group addresses are not in the switch's IGMP-learned table (or, in the absence of IGMP snooping, all multicast frames).

Brooklyn Bridge implements this algorithm in software with one structural change relative to the conventional hardware implementation: the notion of "port" is generalized. A port in Brooklyn Bridge is either a connected client (identified by its session_key) or the special token __LOCAL__ which represents frames originating from or destined for the server's own TAP device. The switch therefore forwards frames to clients identified by their session keys and to the server's local stack via the __LOCAL__ pseudo-port. The forwarding table is a Python dictionary whose keys are MAC addresses (six-byte bytes objects) and whose values are tuples of (port_identifier, timestamp).

7.2 The implementation

Listing 7-1 shows the SwitchPlane class in its essentials. The class exposes three methods: learn() records a source MAC against a port; lookup() retrieves the port for a given destination MAC, with side-effecting eviction of stale entries; and a configuration parameter mac_age_s controls how long entries remain in the table after their last observation.

Listing *7-1. The SwitchPlane class (vpn_server.py).*

```
class SwitchPlane:
    def __init__(self, flood_unknown: bool, mac_age_s: float):
        self.flood_unknown = flood_unknown
        self.mac_age_s = mac_age_s
        self.table: Dict[bytes, Tuple[str, float]] = {}
```

```
        self.lock = threading.Lock()

    def learn(self, mac: bytes, port: str):
        with self.lock:
            self.table[mac] = (port, now())

    def lookup(self, mac: bytes) -> Optional[str]:
        with self.lock:
            entry = self.table.get(mac)
            if entry is None:
                return None
            port, t = entry
            if now() - t > self.mac_age_s:
                del self.table[mac]
                return None
            return port
```

The implementation is approximately twenty lines of Python and is sufficient to handle the entire switching plane of a small Layer-2 network. The brevity is deliberate: the switching algorithm is genuinely simple, and an implementation that obscures this simplicity with elaborate optimizations or premature abstractions does its readers a disservice. Brooklyn Bridge's SwitchPlane is short enough to be read in less than a minute and modified in less than ten.

7.3 MAC aging

The aging behavior is worth explicit discussion. The SwitchPlane evicts entries that have not been observed within mac_age_s seconds (default 300, configurable). The eviction is performed lazily, on lookup, rather than through a periodic sweep. Entries that are not looked up are not evicted; the table can therefore grow unbounded if the bridge encounters a continuous stream of one-time MAC addresses, which is a theoretical pathology that does not arise in practice with realistic traffic.

The aging interval has operational consequences. A short aging interval (say, thirty seconds) causes the table to forget hosts that have been silent for half a minute. If such a host then sends a frame, its source MAC will be relearned without operational impact; if the host receives a frame from another bridge participant, the destination MAC will be unknown and the frame will be flooded. Frequent flooding wastes bandwidth and is undesirable. A long aging interval (say, an hour) avoids this but causes the table to retain stale entries from disconnected hosts; this is largely harmless since stale entries simply produce delivery failures that are corrected by subsequent traffic, but it can create confusion during debugging.

The default of 300 seconds — five minutes — is a compromise drawn from the observation that residential and small-office Layer-2 networks typically see all active hosts within any five-minute window. Operators with unusual traffic patterns may tune the parameter; the trade-off is between flood frequency and stale-entry retention.

7.4 Broadcast and multicast handling

Broadcast and multicast frames present a particular challenge in any Layer-2 network. By definition, they are not destined for a single host; they must be replicated to every interested receiver. In a hardware Ethernet switch, the replication is performed at line rate by dedicated forwarding silicon and imposes minimal cost. In a software switch implementing the same semantics, the replication is performed by the central processor and competes with all other workloads on the host. A misbehaving source — one which is generating broadcast traffic at rates above the bridge's processing capacity — can therefore consume disproportionate bridge resources and starve legitimate traffic.

This pathology is sometimes called a broadcast storm and is a well-known failure mode of Layer-2 networks. Hardware switches typically implement broadcast suppression mechanisms — storm control — that drop broadcasts above a configured rate threshold; the same mitigation is appropriate in a software switch, and Brooklyn Bridge implements it through the TokenBucket class.

Two token buckets are maintained. The first is a global bucket whose rate parameter (default 200 broadcast/multicast packets per second) and burst parameter (default 500 packets) cap the aggregate broadcast traffic across the entire bridge. The second is a per-client bucket whose rate parameter (default 50 packets per second) and burst parameter (default 100 packets) cap the broadcast traffic originated by any single connected client. The combined effect is that no single client can saturate the bridge's broadcast budget, and the bridge as a whole is bounded against broadcast storms regardless of source.

Listing *7-2. The TokenBucket class (vpn_server.py).*

```
class TokenBucket:
    def __init__(self, rate_per_sec: float, burst: float):
        self.rate = rate_per_sec
        self.burst = burst
        self.tokens = burst
        self.last = now()
        self.lock = threading.Lock()

    def allow(self, cost: float = 1.0) -> bool:
        if self.rate <= 0:
            return True  # rate <=0 disables the bucket entirely
        with self.lock:
            t = now()
            elapsed = t - self.last
            self.last = t
            self.tokens = min(self.burst, self.tokens + elapsed * self.rate)
            if self.tokens >= cost:
                self.tokens -= cost
                return True
            return False
```

The implementation is approximately fifteen lines and follows the standard token-bucket algorithm: tokens accumulate at a fixed rate up to a burst cap; each frame consumes a token; frames presented when

the bucket is empty are denied. The denial in Brooklyn Bridge's broadcast path takes the form of frame suppression: the frame is not forwarded to the client whose bucket is empty, with a counter increment recording the drop. The suppression is per-direction: a client whose own outgoing bucket is empty cannot originate further broadcasts, but it continues to receive broadcasts from other clients (subject to the global bucket and the recipient's own per-client bucket on the inbound side).

> **Design choice** *The decision to enforce per-client and global rate limits independently rather than only one or the other is intentional. A per-client-only limit permits N clients each at the per-client cap to collectively exceed any reasonable aggregate limit. A global-only limit permits one misbehaving client to consume the entire global budget and starve well-behaved clients. The combination of both constraints prevents either failure mode.*

8. IP Address Management and Sessions

This chapter examines the IP address management subsystem and the session model that supports it. Section 8.1 reviews the role of IPAM within an overlay network. Section 8.2 covers the session model, with particular attention to the session_key versus client_id distinction that was introduced in version 3.1.1 to prevent identity-spoofing. Section 8.3 walks through the SQLite-backed IPAM implementation. Section 8.4 examines lease semantics.

8.1 The role of IPAM in an overlay network

Layer-2 connectivity does not require IP addressing. Hosts attached to a flat Ethernet broadcast domain can address one another by MAC address and can establish IP communication using whatever IP configuration each host's stack has independently arrived at — through static configuration, external DHCP, or link-local addressing. An overlay network that provides only Layer-2 semantics is therefore not strictly required to provide IP address management.

In practice, however, operators benefit from a centralized IPAM. The benefits are several. First, it relieves operators from configuring each connected host individually with a non-conflicting static address. Second, it provides a single point at which the IP-level structure of the overlay can be observed and managed. Third, it enables the bridge server to push gateway and DNS information to clients, which is useful when the overlay is intended as the clients' default route. Brooklyn Bridge therefore implements IPAM as an optional facility: clients may receive IP configuration from the bridge or may opt out of receiving it via the apply_ip_config configuration flag.

8.2 The session model

Each connection to the bridge produces a session. The session is the unit of state that the server maintains for a connected client; it owns the underlying socket, the framed message stream, the receive and transmit threads, the per-client token bucket, and the IPAM lease. The session is identified by a session_key that the server generates at connection time and that is unique across all sessions the server has ever produced (with overwhelming probability).

Listing 8-1 shows the session_key generation. The format is the client's IP address, the client's TCP source port, and the millisecond timestamp at which the connection was accepted, joined by colons. Two connections cannot collide unless they originate from the same IP and the same source port within the same millisecond — a circumstance that in practice does not occur, because the kernel's source-port allocation prevents reuse of an in-use port.

Listing *8-1. Session key generation in ClientSession (vpn_server.py).*

```
class ClientSession:
    def __init__(self, sock, addr, cfg, counters, probe=None):
        self.sock = sock
```

```
        self.addr = addr
        self.cfg = cfg
        self.counters = counters
        self.probe = probe

        self.session_key = f"{addr[0]}:{addr[1]}:{int(now()*1000)}"
        self.client_id = "unknown"
        self.client_version = "unknown"
        self.last_seen = now()
```

The session_key is distinct from the client_id. The client_id is a label the client sends in its HELLO message; it is operator-chosen, may be any string, and is used for log messages, diagnostic display, and the IPAM persistence record. The session_key is server-generated, ephemeral, and is the identifier against which the IPAM allocates addresses, the SwitchPlane records MAC entries, and the TokenBucket maintains per-client state.

The rationale for distinguishing session_key from client_id

The distinction was introduced in version 3.1.1 in response to an identity-spoofing pathology that existed in earlier versions. In those versions, IPAM allocations were keyed on client_id; a connecting client received the same IP address it had previously received, identified by the client_id it announced. The intended behavior was reasonable — a client reconnecting after a network blip should be able to recover its previous configuration — but the implementation had an exploitable property: a malicious client that learned another client's client_id could connect with the same client_id and receive the legitimate client's IP allocation. The legitimate client's session would then be terminated when the duplicate session displaced it.

The fix was to key IPAM allocations on session_key rather than client_id. Each connection produces a fresh session_key and therefore a fresh IPAM allocation; an attacker who learns another client's client_id gains no advantage. The cost of the fix is that a legitimate client which disconnects and reconnects also receives a fresh allocation — its previous IP address remains leased, against the previous session_key, until the lease expires. In high-churn deployments this can deplete the IP pool more quickly than expected. The mitigation is to shorten the lease interval, not to revisit the keying decision.

> **Design choice** *The session_key and client_id distinction is a special case of a general security pattern: when a system must support both identity and reconnection, the identity should be derived from a quantity the user cannot forge (a session token, a public key, a bearer credential), and the reconnection convenience should be a separate concern. Conflating identity with reconnection convenience is a common error. Brooklyn Bridge's earlier versions made this error; version 3.1.1 corrected it.*

8.3 The SQLite-backed IPAM

The IPAM persists allocations to a SQLite database whose path is configured via the ipam.sqlite_db parameter. The database contains a single table, vpn_clients, with columns for client_key (the

session_key), client_id (the announced label), ip (the allocated IP address as a string), exp (the lease expiration as a Unix timestamp), and last_seen (the most recent observation time). Listing 8-2 shows the allocation routine in essential form.

Listing *8-2. IP address allocation (vpn_server.py).*

```
def allocate(self, client_key: str, client_id: str) -> str:
    with self.lock:
        t = now()
        # Reuse existing unexpired lease if any
        cur = self.conn.execute(
            "SELECT ip, exp FROM vpn_clients WHERE client_key = ?",
            (client_key,))
        row = cur.fetchone()
        if row is not None:
            ip, exp = row
            if exp > t:
                self._upsert(client_key, client_id, ip, t)
                return ip

        # Build the set of IPs in use by other unexpired leases
        cur = self.conn.execute(
            "SELECT ip FROM vpn_clients WHERE exp > ?", (t,))
        used = {r[0] for r in cur.fetchall()}

        # Walk the pool in order, take the first unused address
        start = ipaddress.ip_address(self.pool_start)
        end = ipaddress.ip_address(self.pool_end)
        candidate = start
        while candidate <= end:
            s = str(candidate)
            if s not in used:
                self._upsert(client_key, client_id, s, t)
                return s
            candidate += 1
        raise RuntimeError("IP pool exhausted")
```

The allocation algorithm is straightforward: if an unexpired lease exists for the client_key, return that lease (refreshing it); otherwise, walk the configured IP pool in order, return the first address not currently leased to any other unexpired client_key. If no address is available, raise an exception. The exception is caught by the calling code and converted to an ERROR message to the client, which terminates its session.

8.4 Lease semantics

Each allocation is recorded with an expiration timestamp computed as the time of allocation plus the configured lease_seconds (default 86400, or 24 hours). An allocation persists in the database until its expiration; the SwitchPlane and TokenBucket state, by contrast, are kept only in memory and are lost on server restart. The persistence asymmetry is intentional: an IP address allocation is a relatively scarce

resource that should survive transient outages, whereas a MAC table entry is cheap to regenerate from the next frame and a token bucket's burst credit is cheap to grant.

The lease interval interacts with the session-key keying policy described above. A client which disconnects within the lease interval and reconnects with the same session_key would, in principle, receive its prior allocation; in practice the session_key is regenerated on every connection, so a reconnecting client receives a new key, a new allocation, and a new IP address. The previous IP remains leased until its expiration. Operators concerned about pool exhaustion in high-churn environments should reduce lease_seconds; a value of 600 (ten minutes) is appropriate for environments where reconnections are frequent and the IP pool is small.

9. Source Walkthrough I: Server Top-Down

This chapter begins the structural walkthrough of the Brooklyn Bridge source code. The walkthrough is structural rather than line-by-line: it traces the major paths through the implementation and reproduces the key listings inline, but does not duplicate the entire source. Readers should consult the version 3.1.2 distribution for the complete code; the present chapter is a guided reading.

9.1 File organization

The reference implementation consists of two principal source files. vpn_server.py contains the server: roughly 970 lines including imports, helpers, and the main entry point. vpn_client.py contains the client: roughly 770 lines with similar organization. A small number of supporting files — start_ngrok.bat, tunnel-up.cmd, server_v3.ini, client_v3.ini — provide deployment scaffolding but contain no logic of significance. The test harness, test_vpn_integration.py, contains 42 integration tests across approximately 600 lines and is documented separately in Chapter 12.

9.2 Module structure of vpn_server.py

vpn_server.py is organized in approximately the following order: standard-library imports; conditional Win32 imports; protocol constants; small utility helpers; the TokenBucket class; the ServerConfig dataclass; the load_config function; the Counters class for operational metrics; the FramedSocket class; the TapDevice class; the SQLiteIPAM class; the SwitchPlane class; the ClientSession class; the VPNServerV311 orchestrator class; and the main() entry point. Each section is preceded by a comment banner that delineates its boundaries; the visual organization is intentional and supports navigation.

The order is bottom-up: each component appears after its dependencies. The Counters class, for example, appears before FramedSocket because FramedSocket consumes a Counters instance for its statistics. The ClientSession class appears after FramedSocket, TapDevice, SQLiteIPAM, SwitchPlane, and TokenBucket because it consumes all of them. The orchestrator appears last because it consumes everything. A reader who proceeds linearly through the file therefore encounters each abstraction at the point where it is needed.

9.3 Configuration loading

Configuration is loaded by load_config(), which parses an INI file into a ServerConfig dataclass. The function uses Python's standard configparser module and provides defaults for every parameter. Listing 9-1 shows the structure of ServerConfig in abbreviated form; the full dataclass has approximately thirty fields covering all configurable aspects of the server.

Listing *9-1. The ServerConfig dataclass (abbreviated, vpn_server.py).*

```
@dataclass
```

```
class ServerConfig:
    listen_host: str
    listen_port: int
    max_clients: int
    tap_name: str
    tap_read_buf: int
    tap_use_overlapped: bool
    max_frame_bytes: int
    ethernet_frame_max: int
    flood_unknown_unicast: bool
    mac_age_seconds: float
    heartbeat_interval_s: float
    client_timeout_s: float
    bcast_rate_pps: float
    bcast_burst: float
    bcast_per_client_pps: float
    bcast_per_client_burst: float
    ipam_pool_start: str
    ipam_pool_end: str
    ipam_gateway: str
    ipam_mask: str
    ipam_dns: str
    ipam_lease_seconds: int
    ipam_db_path: str
    # ... debug and pcap parameters ...
```

The dataclass approach to configuration has several practical advantages over a dictionary or a namespace object. Type hints document the intended types of each parameter, supporting static analysis and IDE assistance. Default values are explicit and centralized. The dataclass is immutable in spirit (though not enforced as frozen in the current implementation), which simplifies reasoning about configuration changes during execution. New parameters can be added without changing the load_config signature.

9.4 The orchestrator: VPNServerV311

The VPNServerV311 class is the server's principal orchestrator. Its __init__ method instantiates each major component — FramedSocket-producing factory state, TapDevice, SQLiteIPAM, SwitchPlane, the global TokenBucket, the Counters — and prepares the threads that will be started by start(). Listing 9-2 shows __init__ in abbreviated form.

Listing *9-2. VPNServerV311.__init__ (abbreviated, vpn_server.py).*

```
class VPNServerV311:
    def __init__(self, cfg: ServerConfig, probe=None):
        self.cfg = cfg
        self.probe = probe
        self.counters = Counters()

        self.tap = TapDevice(cfg.tap_name,
                             read_buf=cfg.tap_read_buf,
```

```
                              use_overlapped=cfg.tap_use_overlapped)

        self.ipam = SQLiteIPAM(
            db_path=cfg.ipam_db_path,
            pool_start=cfg.ipam_pool_start,
            pool_end=cfg.ipam_pool_end,
            lease_seconds=cfg.ipam_lease_seconds)

        self.switch = SwitchPlane(
            flood_unknown=cfg.flood_unknown_unicast,
            mac_age_s=cfg.mac_age_seconds)

        self.global_bcast_bucket = TokenBucket(
            cfg.bcast_rate_pps, cfg.bcast_burst)

        self.clients_by_key: Dict[str, ClientSession] = {}
        self.clients_lock = threading.Lock()

        self.listen_sock: Optional[socket.socket] = None
        self.stop_event = threading.Event()

        self.tap_rx_thread = threading.Thread(target=self._tap_rx_loop,
daemon=True)
        self.tap_tx_thread = threading.Thread(target=self._tap_tx_loop,
daemon=True)
        self.accept_thread = threading.Thread(target=self._accept_loop,
daemon=True)
        self.house_thread  = threading.Thread(target=self._house_loop, daemon=True)
```

The initialization is mechanical: each component is instantiated with its configuration parameters and stored as an attribute. The threads are created but not started; the start() method is responsible for the actual launch. The clients_by_key dictionary is the canonical registry of connected clients, keyed by session_key, and the clients_lock provides mutual exclusion for the dictionary's modifications.

9.5 The accept loop

The _accept_loop thread is responsible for accepting incoming connections from the listening socket and creating a new ClientSession for each. Listing 9-3 shows the loop in essential form.

Listing *9-3. The accept loop (vpn_server.py).*

```
def _accept_loop(self):
    while not self.stop_event.is_set():
        try:
            sock, addr = self.listen_sock.accept()
        except OSError:
            break
        try:
            sock.settimeout(None)
            sess = ClientSession(sock, addr, self.cfg, self.counters, self.probe)
```

```
            with self.clients_lock:
                self.clients_by_key[sess.session_key] = sess
            sess.start_threads()
            sess.handshake_thread = threading.Thread(
                target=self._handle_client, args=(sess,), daemon=True)
            sess.handshake_thread.start()
        except Exception as e:
            self.warn(f"accept_setup_failed err={e}")
```

The pattern is straightforward: accept a connection, instantiate a ClientSession, register the session under its session_key, start the session's receive and transmit threads, and dispatch the handshake to a separate thread. The handshake thread invokes _handle_client(sess), which reads the HELLO message, performs IPAM allocation, and sends the CONFIG response.

9.6 The TAP receive loop

The _tap_rx_loop thread reads frames from the local TAP device and presents them to the switch plane. Listing 9-4 shows the loop after the version 3.1.2 corrections; the prior version had a defect in which runt frames could pollute the MAC table, which is now corrected by performing the runt check before the learn() call.

Listing *9-4. The TAP receive loop (vpn_server.py, v3.1.2 corrected).*

```
def _tap_rx_loop(self):
    while not self.stop_event.is_set():
        frame = self.tap.read_frame()
        if not frame:
            continue
        self.counters.inc("tap.rx_frames")

        if not self._clamp_and_count(frame, "tap"):
            continue

        # v3.1.2: Runt check BEFORE learning. Previously a 12-13 byte
        # frame would have its src learned as __LOCAL__ from frame[6:12]
        # and then get dropped as a runt at the next stage — polluting
        # the switch table with MACs from malformed frames.
        if len(frame) < 14:
            self.counters.inc("drops.tap_runt")
            continue

        # Server local ingress => learn as __LOCAL__
        src = frame[6:12]
        self.switch.learn(src, "__LOCAL__")

        self._switch_ingress_from_tap(frame)
```

Each iteration reads a frame from the TAP, increments the receive counter, validates the frame size against the configured maximum, performs the runt check, learns the source MAC against the __LOCAL__

pseudo-port, and presents the frame to the switching logic. The actual dispatch — broadcast to all clients, multicast with rate limiting, or unicast to a known destination — is performed in _switch_ingress_from_tap, which the loop calls.

9.7 The housekeeping loop

The _house_loop thread runs at low frequency (typically once per second) and performs administrative tasks that should not be performed inline with traffic: detecting clients whose last_seen timestamp exceeds client_timeout_s and evicting them; logging periodic statistics if debug logging is enabled; and flushing pcap files if pcap capture is enabled. The loop is short and unremarkable, but its existence is operationally important: a client which has lost connectivity but has not yet generated a TCP RST will continue to occupy a session slot indefinitely without the housekeeper's attention.

10. Source Walkthrough II: ClientSession and Wire Loop

This chapter examines the ClientSession class and the per-client receive and transmit loops it owns. ClientSession is the most operationally significant class in the server: it encapsulates all state for a single connected client, owns the threads that handle that client's traffic, and is the principal locus of per-client policy enforcement.

10.1 ClientSession ownership

Each ClientSession owns the following resources: the underlying TCP socket; the FramedSocket wrapper around that socket; a per-client TokenBucket for outbound broadcast rate limiting; a Counters subset for per-client statistics; the receive and transmit threads dedicated to this client; and a transmit queue (a bounded thread-safe queue) on which other components — principally the SwitchPlane via the server's flood logic — enqueue frames for transmission to this client. The session holds the IPAM-allocated IP address, the announced client_id, and the ephemeral session_key.

Ownership in this context means responsibility for cleanup. When a session terminates — whether by a graceful close from the client, by a server-initiated eviction, or by an exception in one of the threads — the session's cleanup logic closes the socket, signals the transmit queue, joins the threads (or marks them as daemon for non-blocking shutdown), and de-registers the session from the server's clients_by_key dictionary. The cleanup is idempotent: a second invocation has no additional effect, which simplifies error-recovery paths that may invoke cleanup from multiple paths.

10.2 The receive loop

The session's receive loop, _rx_loop, is the principal consumer of inbound traffic from the client. It reads framed messages from the FramedSocket, dispatches them by message type, and updates the last_seen timestamp on each successful read. Listing 10-1 shows the loop in essential form.

Listing *10-1. ClientSession._rx_loop (abbreviated, vpn_server.py).*

```
def _rx_loop(self):
    try:
        while not self.stop.is_set():
            msg = self.fs.recv_msg()
            if msg is None:
                break
            t, payload = msg
            self.last_seen = now()

            if t == MSG_DATA_ETHERNET:
                self.counters.inc("rx.ethernet")
                self._on_ethernet_from_client(payload)
            elif t == MSG_HEARTBEAT:
```

```
                self.counters.inc("rx.heartbeat")
            elif t == MSG_HELLO:
                # already handled in handshake; late HELLO ignored
                self.counters.inc("rx.hello_late")
            elif t == MSG_STATS:
                self.counters.inc("rx.stats")
            else:
                self.counters.inc("rx.unknown_type")
    except Exception as e:
        self.warn(f"rx_loop err={e}")
    finally:
        self.shutdown()
```

The dispatch is a flat sequence of type checks. Each message type increments a counter and invokes its handler. The dominant case in production traffic is MSG_DATA_ETHERNET, which carries an Ethernet frame and is routed through _on_ethernet_from_client(). The other types are handled inline because their handling is trivial.

The finally clause ensures that shutdown() is called regardless of how the loop exits. This is important: an exception in _on_ethernet_from_client (an unlikely event but possible) must not leave the session registered with the server. The shutdown path closes the socket, sets the stop flag, and de-registers the session.

10.3 The Ethernet handler

_on_ethernet_from_client is the per-client ingress point for traffic the client is sending across the bridge. It validates the frame, learns the source MAC into the switch plane against this session's session_key, and dispatches the frame according to its destination MAC.

Listing *10-2. _on_ethernet_from_client (vpn_server.py).*

```
def _on_ethernet_from_client(self, frame: bytes):
    if not self.server._clamp_and_count(frame, "client"):
        return
    if len(frame) < 14:
        self.counters.inc("drops.client_runt")
        return

    dst = frame[0:6]
    src = frame[6:12]
    self.server.switch.learn(src, self.session_key)

    if is_broadcast_mac(dst) or is_multicast_mac(dst):
        # Per-client and global rate limits both apply
        if not self.bcast_bucket.allow():
            self.counters.inc("drops.bcast_per_client_rate")
            return
        if not self.server.global_bcast_bucket.allow():
            self.counters.inc("drops.bcast_global_rate")
```

```
            return
        self.server._flood_to_clients_except(self, frame)
        self.server._enqueue_to_local_tap(frame)
    else:
        target = self.server.switch.lookup(dst)
        if target == "__LOCAL__":
            self.server._enqueue_to_local_tap(frame)
        elif target is None:
            if self.server.cfg.flood_unknown_unicast:
                self.server._flood_to_clients_except(self, frame)
                self.server._enqueue_to_local_tap(frame)
            else:
                self.counters.inc("drops.unknown_unicast")
        else:
            with self.server.clients_lock:
                tgt_sess = self.server.clients_by_key.get(target)
            if tgt_sess is not None:
                self.server._enqueue_to_client(tgt_sess, frame)
            else:
                # learned port no longer exists — fall back to flood
                self.server._flood_to_clients_except(self, frame)
                self.server._enqueue_to_local_tap(frame)
```

The handler is the heart of the bridge. Each Ethernet frame passes through it precisely once. The dispatch decision — whether to flood, send to local TAP, send to a specific client, or drop — is made on the basis of the destination MAC's classification and the switch plane's lookup result. The fallback path, in which the learned port no longer exists, handles the unusual case of a client disconnecting between the lookup and the enqueue; the frame is treated as if the destination were unknown.

10.4 The transmit loop

The session's transmit loop, _tx_loop, is the principal producer of outbound traffic to the client. It dequeues frames from the transmit queue, wraps each frame in a DATA_ETHERNET message, and writes the wrapped bytes to the FramedSocket. The loop is short and unremarkable; the queueing logic that feeds it (in _enqueue_to_client and _flood_to_clients_except) is more interesting because it must handle the case of a slow client whose transmit queue is full.

Listing *10-3. ClientSession._tx_loop and the bounded queue (vpn_server.py).*

```
def _tx_loop(self):
    try:
        while not self.stop.is_set():
            try:
                frame = self.tx_q.get(timeout=0.5)
            except queue.Empty:
                continue
            if frame is None:  # shutdown sentinel
                break
            try:
```

```
                self.fs.send_msg(MSG_DATA_ETHERNET, frame)
                self.counters.inc("tx.ethernet")
            except Exception as e:
                self.warn(f"tx err={e}")
                break
    finally:
        self.shutdown()
```

The transmit queue is bounded (default 2000 entries). When the queue is full and a producer attempts to enqueue, the producer's enqueue raises queue.Full and the frame is dropped with a counter increment. This bounded behavior is operationally important: an unbounded queue would, in the presence of a slow client, accumulate memory indefinitely until the server exhausted available memory and crashed. A bounded queue caps the per-client memory footprint at a known maximum and ensures that one slow client cannot affect the others' service.

11. Source Walkthrough III: Client and Notable Bugs

This chapter examines the client implementation. The client is structurally symmetric to the server: it has the same FramedSocket layer, the same TapDevice wrapper (with platform-specific particulars), and the same threading model with TAP and network sides connected by queues. This chapter focuses on the points at which the client diverges from the server and reviews the five corrections incorporated in version 3.1.2.

11.1 Module structure of vpn_client.py

vpn_client.py is organized similarly to vpn_server.py. The file begins with imports (now including the conditional Win32 imports introduced in version 3.1.2), proceeds through small utilities, defines the ClientConfig dataclass and the load_config function, defines a Counters class identical in structure to the server's, defines its own FramedSocket and TapDevice, defines several netsh-related helpers for applying the assigned IP configuration to the local TAP, and concludes with the VPNClientV311 orchestrator and main() entry point. The total length is approximately 770 lines.

The client does not contain a SwitchPlane, an SQLiteIPAM, or a TokenBucket. These are server-side concerns. The client's role is to relay frames between its local TAP and the server-side bridge; switching decisions are made on the server, not the client. The client therefore has a simpler internal structure: two queues (TAP-to-network and network-to-TAP), four threads (TAP receive, TAP transmit, network receive, network transmit), and a small amount of orchestration code.

11.2 The handshake

The client's handshake is conducted in _connect_run, which is called from start() in a reconnect loop. _connect_run opens a TCP socket to the configured server, sends a HELLO message, waits for a CONFIG response, applies the received IP configuration (if apply_ip_config is enabled), starts the receive and transmit threads, and waits for one of the threads to signal disconnection.

Listing *11-1. The handshake in _connect_run (abbreviated, vpn_client.py).*

```
def _connect_run(self):
    self.info(f"connecting to {self.cfg.server_host}:{self.cfg.server_port}")
    s = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
    s.settimeout(10.0)
    try:
        s.connect((self.cfg.server_host, self.cfg.server_port))
    except Exception as e:
        self.warn(f"connect_failed err={e}")
        try: s.close()
        except: pass
        return
```

```
    s.settimeout(None)
    self.sock = s
    self.fs = FramedSocket(s, self.cfg.max_frame_bytes, role="client")

    # Send HELLO
    hello = {
        "client_id": self.cfg.client_id,
        "version": self.cfg.protocol_version,
        "platform": "windows",
    }
    self.fs.send_msg(MSG_HELLO, json.dumps(hello).encode("utf-8"))

    # Wait for CONFIG
    msg = self.fs.recv_msg()
    if msg is None:
        self.warn("connection closed before CONFIG")
        return
    t, payload = msg
    if t != MSG_CONFIG:
        self.warn(f"expected CONFIG got type={t}")
        return

    cfg_obj = json.loads(payload.decode("utf-8"))
    self.assigned_cfg = cfg_obj
    self.session_key = cfg_obj.get("session_key")
    self._apply_config(cfg_obj)

    # Start net threads, wait for disconnect
    self.net_rx_thread = threading.Thread(target=self._net_rx_loop, daemon=True)
    self.net_tx_thread = threading.Thread(target=self._net_tx_loop, daemon=True)
    self.net_rx_thread.start()
    self.net_tx_thread.start()
```

11.3 Notable bugs and their corrections

Version 3.1.2 incorporated five corrections identified during a structured triage pass. The corrections are listed here in approximate order of operational severity.

11.3.1 Conditional Win32 imports

The original implementation imported win32file, win32con, and pywintypes unconditionally at the top of both vpn_server.py and vpn_client.py. On any host without pywin32 installed — including all Linux and macOS hosts — the import failed at module load, preventing even the protocol layer from being loaded for testing. The version 3.1.2 correction wraps the imports in try/except, sets a _WIN32_OK flag based on success, and defers the failure until TapDevice.open() is called. The protocol layer is therefore portable across platforms; only the actual TAP I/O remains Windows-only.

Listing *11-2. The conditional Win32 import pattern (vpn_server.py and vpn_client.py).*

```
try:
    import win32file
    import win32con
    import pywintypes
    _WIN32_OK = True
    _WIN32_IMPORT_ERROR = None
except ImportError as _e:
    _WIN32_OK = False
    _WIN32_IMPORT_ERROR = _e
    win32file = None
    win32con = None
    pywintypes = None
```

11.3.2 Runt-then-learn in the server's TAP receive loop

The server's _tap_rx_loop, prior to version 3.1.2, performed a length check of len(frame) >= 12 before learning the source MAC, but the runt drop at the next stage required len(frame) >= 14. A frame of 12 or 13 bytes therefore had its supposed source MAC (extracted from frame[6:12]) learned to the __LOCAL__ pseudo-port and was then dropped at the runt check. The result was that the switch table could be polluted with MAC addresses extracted from malformed frames, and these polluted entries could subsequently mis-route legitimate unicast traffic. The correction moves the runt check before the learn, ensuring that no MAC is learned from a frame that will be dropped.

11.3.3 IndexError in is_multicast_mac

The is_multicast_mac helper performed dst[0] & 1 without checking the length of the dst argument. No code path currently fed it empty input — the runt checks above this layer prevented it — but the helper was a latent crash awaiting a future code change that bypassed the upstream checks. The correction length-guards both is_broadcast_mac and is_multicast_mac. An empty bytes object is, by convention, neither broadcast nor multicast.

11.3.4 Slow disconnect detection on Linux

The client's stop() method closed the socket without first calling shutdown(). On Linux, this can leave the connection half-open until kernel buffer pressure forces the FIN through, with the consequence that the server's recv() blocks for several seconds before noticing the client is gone — and during this window the server retains a stale session in clients_by_key. The correction inserts a shutdown(SHUT_RDWR) before the close. The server now observes the disconnect immediately.

Listing *11-3. The corrected client stop() (vpn_client.py).*

```
def stop(self):
    self.stop_event.set()
    try:
        if self.sock:
            # v3.1.2: shutdown before close so the server's recv()
            # returns immediately instead of blocking until kernel
            # buffer pressure forces the FIN through.
```

```
            try:
                self.sock.shutdown(socket.SHUT_RDWR)
            except OSError:
                pass
            self.sock.close()
    except Exception:
        pass
    self.sock = None
    self.fs = None
    self.tap.close()
```

11.3.5 Client-side runt frame forwarding

The client's _clamp_and_count helper, prior to version 3.1.2, only enforced an upper bound on frame size. A runt frame from the local TAP would therefore be wrapped in a DATA_ETHERNET message and sent across the underlay to the server, which would drop it at its own runt check. The wire bandwidth was wasted and the bounds enforcement on client and server were inconsistent. The correction adds a lower-bound check on the client side, dropping runts before they cross the underlay. A new counter, drops.runt_eth, records the rejections.

11.4 Operational consequences of the corrections

Each correction was accompanied by a regression test in the integration suite documented in Chapter 12. The cumulative effect of the corrections was small in absolute terms — the bridge was operational before each correction was applied — but each correction closed a path through which incorrect or surprising behavior could occur under unusual circumstances. The Win32 import correction, in particular, transformed the development experience by permitting non-Windows-based testing and CI.

12. Source Walkthrough IV: The Test Harness

This chapter examines the integration test suite that accompanies the version 3.1.2 reference implementation. The suite consists of forty-two tests organized into thirteen test classes and is implemented in approximately six hundred lines of Python using the pytest framework. The tests run end-to-end on any platform supporting Python 3.10 or later; they require neither pywin32 nor an actual TAP device.

12.1 The harness architecture

The harness is structured around two principal components: a VPNTestEnv class that instantiates a server in the test process, and a ClientHarness class that instantiates a client. Both components substitute a FakeTap class for the production TapDevice; the FakeTap has the same interface (open, close, read_frame, write_frame) but uses in-memory queues rather than the OpenVPN driver. The protocol layer above the TAP is exercised end to end with real TCP traffic on ephemeral 127.0.0.1 ports.

Listing *12-1. The FakeTap substitute (test_vpn_integration.py).*

```
class FakeTap:
    """Substitute for TapDevice: in-memory queue with the same surface area."""

    def __init__(self, name="", read_buf=65536, use_overlapped=False):
        self.name = name
        self.read_buf = read_buf
        self.use_overlapped = use_overlapped
        self.handle = "fake"
        self.closing = threading.Event()
        self._rx = queue.Queue()  # frames the server will read
        self._tx = queue.Queue()  # frames the server has written

    def open(self):
        return

    def close(self):
        self.closing.set()
        self._rx.put(None)  # wake any blocking read

    def read_frame(self):
        try:
            f = self._rx.get(timeout=0.2)
        except queue.Empty:
            return None
        return f

    def write_frame(self, frame: bytes) -> bool:
        self._tx.put(frame)
        return True
```

```
    # Test helpers
    def push_inbound(self, frame: bytes):
        self._rx.put(frame)

    def pop_outbound(self, timeout: float = 1.0):
        try:
            return self._tx.get(timeout=timeout)
        except queue.Empty:
            return None
```

The FakeTap is the central insight of the harness design. By implementing the same interface as TapDevice, it permits the production code paths in VPNServerV311 and VPNClientV311 to execute unmodified; the only deviation from production is the substitution of in-memory queues for actual driver I/O. The protocol layer, the threading, the IPAM, the switch plane — all execute exactly as in production. Only the TAP boundary is mocked.

12.2 The test classes

The forty-two tests are organized into thirteen classes, each exercising a coherent subsystem. The classes are summarized below.

- TestProtocolPrimitives — pure-function tests of the framing struct, broadcast and multicast helpers, and their defensive behavior on edge inputs.
- TestTokenBucket — initial-burst, zero-rate (which disables the bucket), and time-based refill.
- TestIPAM — first allocation, lease reuse, distinct-client allocation, pool exhaustion, client_id label change, persistence across instance recreation.
- TestSwitchPlane — learn and lookup, unknown-MAC behavior, relearn, aging.
- TestServerLifecycle — server binds the configured port, two test environments coexist on different ports.
- TestHandshake — synthetic client over a real socket performs HELLO and receives CONFIG, IP is in the configured pool, two clients receive distinct IPs, server tracks the connected client.
- TestEthernetForwarding — synthetic client sends a frame and the server forwards to the local TAP; a frame on the local TAP is forwarded to the connected client.
- TestProtocolRobustness — garbage payload from one client does not affect another, oversized frames are rejected without crashing the server, clean disconnect is handled correctly.
- TestRuntFrameHandling — server rejects runts at both client and TAP ingress; this class pins the v3.1.2 correction described in Chapter 11.
- TestIPAMReconnectBehavior — documents the v3.1.1 anti-spoofing trade-off, verifies that a reconnecting client receives a fresh IP.
- TestMACLearning — frame from a synthetic client teaches the server, and a later frame from the TAP routes correctly to the originating client.

- TestRealClientHandshake — full VPNClientV311 instance connects, handshakes, and stores its assigned configuration.
- TestRealClientForwarding — real client sends a frame and the server forwards correctly; real client receives a frame from the server's TAP; two real clients exchange a frame via the server's switching plane (the marquee end-to-end test).
- TestRealClientLifecycle — clean disconnect propagates to the server within bounded time; heartbeats from a real client reach the server within their configured interval.

12.3 The marquee test

The most operationally significant test in the suite is test_client_to_client_via_server_switching in TestRealClientForwarding. This test instantiates two real VPNClientV311 instances, connects both to the same server, has them exchange Ethernet frames, and verifies that the server's switching plane routes correctly between them. It is the closest the harness comes to a production-equivalent end-to-end test.

Listing *12-2. The marquee end-to-end test (test_vpn_integration.py).*

```
def test_client_to_client_via_server_switching(self, env):
    """Two real clients: A sends a frame with B's MAC as destination.
    The server should learn A's MAC, route the frame to B, who
    receives it on its TAP."""
    ha = ClientHarness(env, client_id="A")
    hb = ClientHarness(env, client_id="B")
    try:
        ha.wait_for_config(timeout=3.0)
        hb.wait_for_config(timeout=3.0)
        time.sleep(0.2)

        mac_a = b"\x02\x00\x00\xaa\xaa\xaa"
        mac_b = b"\x02\x00\x00\xbb\xbb\xbb"

        # First, B sends a frame so the server learns mac_b → B's session
        frame_from_b = mac_a + mac_b + b"\x08\x00" + b"FROM-B"
        hb.fake_tap.push_inbound(frame_from_b)
        time.sleep(0.3)

        # Confirm server learned mac_b
        sk_b = hb.client.session_key
        assert env.server.switch.lookup(mac_b) == sk_b

        # Drain anything on A's TAP from this learning step
        while ha.fake_tap.pop_outbound(timeout=0.1) is not None:
            pass

        # Now A sends a frame addressed to B's MAC
        frame_from_a = mac_b + mac_a + b"\x08\x00" + b"FROM-A-TO-B"
        ha.fake_tap.push_inbound(frame_from_a)
```

```
        # B should receive it on its TAP. (NOT A — unicast, not flood.)
        got_b = hb.fake_tap.pop_outbound(timeout=2.0)
        assert got_b == frame_from_a

        # A should NOT have received its own frame back
        got_a = ha.fake_tap.pop_outbound(timeout=0.5)
        assert got_a is None
    finally:
        ha.stop()
        hb.stop()
```

The test exercises the full end-to-end path: TAP → client → underlay → server switching → underlay → client → TAP. The verification checks both that the frame arrives at the correct destination and that it does not arrive at the wrong one. The latter check — that A does not receive its own frame back — protects against a class of bug in which the server might inadvertently flood frames back to their origin.

12.4 Test execution

The complete suite runs in approximately thirteen seconds on commodity hardware. Each test gets a fresh temporary directory, a fresh IPAM database, and an ephemeral port; tests do not share state and can be run in any order. The suite is invoked with python3 -m pytest test_vpn_integration.py -v from the project root.

The test harness has proven its value beyond regression protection. The five corrections incorporated in version 3.1.2 were all identified through interactive use of the harness: the runt-then-learn defect was discovered by writing a test that pushed a 12-byte frame onto the fake TAP and observing the polluted switch table; the slow disconnect was identified by writing a test that called stop() and timed how long the server took to free the session. The harness functioned as a debugging tool as much as a regression tool.

13. Deployment

This chapter documents the procedures for deploying Brooklyn Bridge in production. Section 13.1 covers server deployment. Section 13.2 covers client deployment. Section 13.3 surveys alternatives to ngrok as the underlay. Section 13.4 addresses the operationally important question of running the bridge as a Windows service.

13.1 Server deployment

The server requires a Windows host with Python 3.10 or later, the OpenVPN TAP-Windows driver, the pywin32 module, and outbound internet access for the underlay. Local administrator privileges are required for TAP I/O on Windows; the server cannot be run as a non-privileged user. Memory requirements are modest — under 100 megabytes resident with ten connected clients — and CPU requirements scale linearly with broadcast traffic and connection count, with a single core on commodity hardware sufficient for the project's typical operational scale.

The deployment procedure is as follows. First, the operator installs the OpenVPN TAP-Windows driver. The driver is bundled with the OpenVPN community installer; alternative distributions exist but the OpenVPN-bundled driver has the longest production track record. After installation, the operator opens Network Connections, identifies the new TAP adapter, and renames it to match the tap.name parameter in server_v3.ini (default N2NHU-Server-TAP). The operator assigns the TAP a static IP matching the ipam.gateway parameter (default 10.50.50.1) with the configured subnet mask.

Second, the operator places the server source files in a working directory: vpn_server.py, server_v3.ini, and any auxiliary batch files. The INI file is edited as needed; the most commonly tuned parameters are listen_port (the local TCP port on which the server listens), the IPAM pool boundaries, the broadcast rate limits, and the debug logging flags. Default values are reasonable for most deployments.

Third, the operator establishes the underlay. With ngrok as the default, the operator runs ngrok tcp 4444 (substituting the configured listen port) in a persistent terminal session. ngrok prints a public hostname of the form 0.tcp.ngrok.io:NNNNN; this hostname is the address that clients will use as their server_host. Operators using alternative underlays substitute the appropriate setup commands.

Fourth, the operator starts the server with python vpn_server.py --config server_v3.ini. The server logs its banner, opens the TAP, binds the listening socket, and waits for clients. The first connection takes a moment longer than subsequent connections because the IPAM SQLite database is being initialized.

13.2 Client deployment

The client requires a Windows host with the same prerequisites as the server: Python 3.10 or later, the OpenVPN TAP-Windows driver, pywin32, and outbound internet access. Memory and CPU requirements are minimal; the client can run on hardware as small as a single-board computer or a low-cost mini-PC.

The deployment procedure mirrors the server: install the TAP driver, rename the resulting adapter to match the tap.name in client_v3.ini, edit the INI file with the server's underlay address (the ngrok hostname noted above), and start the client with python vpn_client.py --config client_v3.ini. The client logs its handshake; within a few seconds, the assigned IP appears on the TAP adapter and the client appears in the server's clients_by_key dictionary.

Operators deploying multiple clients to remote sites benefit from the zero-touch provisioning pattern. The operator pre-images each mini-PC with the client source, the TAP driver, and an INI file pre-configured with the operator's server address and a unique client_id. The mini-PC is shipped to the customer with instructions to plug in power and Ethernet; on boot, the client starts (via a Windows scheduled task or service), connects to the server, and joins the bridge. The customer is not required to perform any configuration.

13.3 Alternative underlays

Brooklyn Bridge supports any reliable byte-oriented underlay. The default ngrok configuration is convenient but several alternatives are operationally relevant.

Direct TCP between two publicly reachable hosts is the simplest alternative. The operator configures the server's listen_host to a publicly addressable interface and ensures the listen_port is reachable from the internet (typically by configuring the host's firewall and any upstream NAT). Clients use the public IP address and port directly. This eliminates the ngrok dependency and removes the latency of routing through ngrok's edge servers.

Cloudflare Tunnel offers an architecturally similar service to ngrok with a more generous free tier and integrated TLS. The operator configures a Cloudflare Tunnel client (cloudflared) to forward a public hostname to the local listen port. Clients use the assigned hostname. The principal advantage over ngrok is the absence of session limits on the free tier; the principal disadvantage is somewhat more complex initial setup.

An SSH local forward through an intermediate jump host is appropriate when the operator already has SSH infrastructure and prefers to avoid additional cloud dependencies. The bridge's server runs on the operator's host or on a host reachable from a jump host; clients open an SSH session to the jump host with a local forward of the bridge port. Clients then connect to localhost on the forwarded port. This adds a layer of configuration but eliminates dependency on external services.

13.4 Running as a Windows service

In production, Brooklyn Bridge should run as a Windows service to ensure that it starts on boot, restarts on failure, and is decoupled from interactive logon sessions. Two approaches are available: NSSM (the Non-Sucking Service Manager) and a Windows scheduled task running at system start.

NSSM is a small open-source utility that wraps an arbitrary executable as a Windows service with restart-on-failure and logging. The operator installs NSSM, creates a service named (for example) BrooklynBridgeServer that invokes python vpn_server.py --config server_v3.ini in the working directory, and configures the service to restart on failure. NSSM provides a graphical configuration utility for service properties and is the simplest approach for operators not familiar with the Windows Service Control Manager.

A scheduled task at system start with the highest privileges is an alternative for operators who prefer to avoid third-party utilities. The task runs python vpn_server.py with appropriate arguments and is configured with restart-on-failure conditions. The principal disadvantage relative to NSSM is the absence of integrated logging; output must be redirected to log files manually.

14. Configuration Reference

This chapter documents every configuration parameter for both server and client. The parameters are grouped by INI section. Defaults reflect version 3.1.2 of the reference implementation.

14.1 Server INI: [server]

Parameter	Default	Description
`listen_host`	`0.0.0.0`	Address on which the server binds. 0.0.0.0 listens on all interfaces; 127.0.0.1 limits binding to the loopback (appropriate when the underlay is co-located with the server, as with ngrok on the same host).
`listen_port`	`4444`	TCP port on which the server listens. The underlay (ngrok or alternative) is configured to forward to this port.
`max_clients`	`50`	Maximum concurrent connected clients. The server refuses additional connections once the cap is reached.

14.2 Server INI: [tap]

Parameter	Default	Description
`name`	`N2NHU-Server-TAP`	Friendly name of the TAP adapter as shown in Network Connections. The operator must rename the OpenVPN-installed adapter to match this value, or set this value to the GUID of the desired adapter.
`read_buf`	`65536`	Size of the buffer used for ReadFile calls against the TAP device.
`use_overlapped`	`false`	Whether to use Windows overlapped (asynchronous) I/O for TAP. Synchronous I/O is the default because it is more reliable across the range of TAP driver versions encountered in the field.

14.3 Server INI: [transport]

Parameter	Default	Description
`max_frame_bytes`	`262144`	Hard upper bound on the size of any wire-protocol message. Defends against adversarial size declarations in the framing header.
`ethernet_frame_max`	`1600`	Hard upper bound on Ethernet frame size. Frames larger than this are dropped before being tunneled. Protects the underlay against unexpected jumbo frames.
`heartbeat_interval_seconds`	`5`	Interval at which clients send heartbeats. The server's idle-detection logic uses this value indirectly through client_timeout_seconds.
`client_timeout_seconds`	`30`	Time without any traffic from a client before the server declares the session dead. Should be at least 3x the heartbeat interval.

14.4 Server INI: [mac_learning]

Parameter	Default	Description
`flood_unknown_unicast`	`true`	If true, frames addressed to unknown unicast MACs are flooded to all ports (standard switch behavior). If false, they are dropped.
`mac_age_seconds`	`300`	Time after which an unrefreshed MAC table entry is evicted on its next lookup. Lower for highly dynamic environments; higher for stable ones.

14.5 Server INI: [broadcast]

Parameter	Default	Description
`global_pps`	`200`	Maximum aggregate broadcast and multicast packets per second across the entire bridge.
`global_burst`	`500`	Burst capacity of the global token bucket. Permits short-term excess above the steady-state rate.
`per_client_pps`	`50`	Maximum broadcast/multicast packets per second from any single client.
`per_client_burst`	`100`	Burst capacity of the per-client token bucket.

14.6 Server INI: [ipam]

Parameter	Default	Description
`pool_start`	`10.50.50.50`	First IP address available for allocation.
`pool_end`	`10.50.50.250`	Last IP address available for allocation.
`gateway`	`10.50.50.1`	Gateway address pushed to clients. Should match the IP statically assigned to the server's TAP adapter.
`subnet_mask`	`255.255.255.0`	Subnet mask pushed to clients.
`dns`	`(=gateway)`	DNS server pushed to clients.
`lease_seconds`	`86400`	Duration for which an IP address remains bound to a session_key after allocation.
`sqlite_db`	`vpn_ipam.db`	Path to the SQLite database that persists IPAM allocations.

14.7 Client INI overview

The client INI follows the same general structure as the server INI. The principal sections are [client] (server connection details and identity), [tap] (local TAP adapter), [transport] (frame size limits and timeouts, mirroring the server), [apply] (whether to apply the assigned IP configuration to the local TAP via netsh), [routes] (additional routes to install through the bridge), and [debug]/[pcap] (diagnostic logging). Operators rarely need to modify the client INI beyond the [client] section after initial deployment.

15. Use Cases

This chapter examines the operational scenarios for which Brooklyn Bridge has been deployed or is well-suited. Five scenarios are presented in detail. The intention is not to cover the full range of possible applications but to illustrate, through concrete cases, the kinds of operational problem to which a small Layer-2 bridge is the appropriate response.

15.1 Industrial pump station remote diagnostics

The triggering case described in Chapter 2 — the pump station with the multicast-discovering PLC — is the canonical application of Brooklyn Bridge. The customer's diagnostic tool runs on a workstation in the operator's office; the PLC is located at a remote pump station behind a cellular gateway with carrier-grade NAT. The operator's existing remote access tooling — a Layer-3 VPN — delivers IP packets between office and pump station but does not preserve the multicast announcements on which the diagnostic tool's PLC discovery depends.

Brooklyn Bridge bridges the gap. A small mini-PC at the pump station runs the bridge client; a server at the operator's office runs the bridge server. The pump station's PLC and the office workstation are both attached to their respective sides of the bridge through their local Ethernet networks. Multicast announcements from the PLC traverse the bridge and reach the workstation; the workstation's diagnostic tool discovers the PLC as if it were on the local network, and configuration sessions proceed normally. The deployment requires no modification to the PLC, no specialized configuration of the cellular gateway, and no change to the operator's existing tooling.

The throughput requirement is modest — the multicast announcements are infrequent, and the configuration sessions transfer small amounts of data — so the TCP-on-TCP underlay is not a limitation. The encryption requirement is met by ngrok's TLS, which the operator considers adequate for a non-public-safety industrial deployment. The total latency added by the bridge is small relative to the inherent latency of cellular connectivity, which dominates the user experience.

15.2 Multi-site SD-WAN extension

Several of the operator's customers operate multiple sites that, prior to Brooklyn Bridge, were connected by site-to-site VPNs at Layer 3. The customers' applications increasingly include protocols that depend on Layer-2 semantics — Windows network neighborhood discovery, mDNS-based service registration, IP camera multicast streams — and the existing VPNs do not deliver these protocols across the inter-site boundary.

A bridge server at each customer's primary site, with bridge clients at remote sites, extends a single broadcast domain across the customer's entire footprint. Hosts at remote sites appear in the primary site's network neighborhood; service discovery functions across sites; multicast streams reach all interested

receivers. The operator continues to maintain the underlying Layer-3 VPN for bandwidth-intensive traffic that does not benefit from Layer-2 semantics; the bridge handles only the protocols that require flat broadcast.

15.3 Educational laboratory deployment

Brooklyn Bridge has been deployed in a college networking course to permit students to experiment with Layer-2 protocols in an environment where they otherwise would not have administrative access to a physical Ethernet switch. Each student receives a virtual machine with a bridge client preconfigured to connect to a course server; the entire class joins the same bridge and forms a single broadcast domain spanning the students' VMs.

The pedagogical advantages are several. Students can observe ARP traffic in action by capturing on their VM and watching the broadcasts from other students' VMs propagate. Students can experiment with rogue DHCP servers, malformed broadcasts, and other Layer-2 misbehaviors in an isolated environment. The instructor can introduce specific topology constraints by configuring per-client rate limits or by selectively disconnecting clients during exercises. The bridge's source code itself becomes course material; students extend the bridge to support new message types or implement specific switch behaviors as projects.

15.4 Industrial control system extensibility

Industrial environments commonly include legacy equipment that speaks Layer-2-only protocols and cannot be retrofitted to support Layer-3 routing. The most common examples are programmable logic controllers from the 1990s and early 2000s, building automation systems based on BACnet, and laboratory instrumentation that uses GVCP (the GigE Vision Control Protocol) for camera discovery and configuration. Each of these depends on Layer-2 broadcasts or multicasts that do not cross routed boundaries.

Brooklyn Bridge permits the integration of such equipment into modern operational networks without modifying the equipment itself. The legacy equipment is attached to a local Ethernet segment; a small bridge client attached to that segment connects to a bridge server in the modern operations center; modern monitoring and control software at the operations center addresses the legacy equipment as if it were on a local network. The legacy equipment continues to operate without modification and without awareness of the bridge.

15.5 Hobbyist and homelab interconnection

Hobbyist deployments — homelabs, distributed gaming, retro-computing networks — frequently benefit from Layer-2 connectivity for the same reasons as commercial deployments: service discovery, link-local addressing, multicast applications. The hobbyist scale is well-matched to Brooklyn Bridge's operational profile: small numbers of connected hosts, modest bandwidth requirements, a strong preference for self-

hosted infrastructure over commercial cloud services. The bridge is sometimes deployed for exactly such reasons by operators who would not otherwise be candidates for an SD-WAN product.

A representative deployment links several friends' homelabs into a single broadcast domain. Each participant runs a bridge client on a small home server; one participant runs a bridge server on a host with publicly reachable connectivity. Game servers running on participants' hardware appear to one another by their LAN-discovery announcements; file shares advertised by SMB are visible across the bridge; participants can experiment with Active Directory, domain controllers, and similar enterprise-style infrastructure in an environment where Layer-2 semantics function correctly. The deployment is operationally simple and costs nothing beyond the participants' existing hardware and internet connectivity.

16. Pedagogical Applications

Brooklyn Bridge's design intent included pedagogical use. The implementation is small enough to be read end-to-end by an undergraduate student in a single sitting; it illustrates concepts in software engineering — concurrency, length-prefixed framing, lease-based resource allocation — alongside concepts in networking; and it produces observable effects that students can verify experimentally. This chapter surveys the pedagogical exercises for which the bridge has been used and proposes additional exercises that an instructor adopting the bridge as course material may wish to consider.

16.1 Exercises in observation

The simplest exercises require students only to observe the bridge's behavior in a controlled environment. Each exercise produces an experimentally verifiable result that connects to a theoretical concept covered in lecture.

Exercise 1: ARP propagation across the bridge. Students attach two virtual machines to a bridge instance, configure each VM with an IP address in the bridge's subnet, and use Wireshark on each VM to capture traffic. One VM pings the other; students identify the ARP request broadcast from the originating VM, verify that it appears in the capture on the other VM, and observe the ARP reply unicast in return. The exercise demonstrates that the bridge correctly preserves Layer-2 broadcast semantics.

Exercise 2: MAC learning observed via the SwitchPlane. Students enable the server's debug logging, attach two VMs, and observe the log entries that appear when each VM sends its first frame. The students identify the MAC addresses being learned, the session_keys against which they are learned, and the moment at which a MAC table entry ages out. The exercise demonstrates that source-learning is a passive, observation-driven process rather than an explicit registration.

Exercise 3: Broadcast rate limiting. Students configure a small per-client rate limit (say, 5 packets per second) and have one VM generate broadcasts at a rate above the limit. They observe, in the server's drop counters, that the excess broadcasts are suppressed. The exercise illustrates the practical operation of the token-bucket algorithm and the rationale for storm control in Layer-2 networks.

16.2 Exercises in modification

Intermediate exercises require students to modify the bridge's source and observe the effects of their modifications. These exercises connect software engineering practice with networking theory.

Exercise 4: Add a new message type. Students extend the wire protocol with a new message type, MSG_PING (0x07), in which the client periodically sends a small payload and the server echoes it back. Students implement the message on both sides, add a counter for ping round-trip times, and observe their implementation in operation. The exercise illustrates protocol extension under a length-prefixed framing scheme.

Exercise 5: Implement IGMP snooping. Students add to the SwitchPlane a multicast group membership table that records which sessions have indicated interest in specific multicast groups by sending IGMP join messages. Students modify the broadcast/multicast forwarding logic to respect the table: multicast frames are sent only to sessions that have joined the appropriate group. The exercise illustrates the optimization that hardware switches commonly perform to reduce unnecessary multicast flooding.

Exercise 6: Add Spanning Tree Protocol awareness. Students extend the bridge to detect and respond to BPDUs (Bridge Protocol Data Units), the messages by which physical Ethernet switches negotiate spanning tree to prevent loops. The exercise illustrates the historical context of why Ethernet bridges require loop-prevention and why software bridges typically do not implement STP — they are more often deployed in topologies where loops are structurally impossible.

16.3 Exercises in construction

Advanced exercises require students to construct related systems from scratch using Brooklyn Bridge as reference. These exercises are appropriate for capstone projects or research-level coursework.

Exercise 7: A Linux client. Students implement a Linux client with the same protocol layer as vpn_client.py but using Linux's /dev/net/tun interface instead of Windows TAP. The protocol layer is platform-independent and can be reused; only the TAP wrapper requires reimplementation. The completed client connects to the existing Windows server and interoperates correctly. The exercise illustrates how good abstraction at the protocol layer enables platform portability.

Exercise 8: A UDP-underlay variant. Students replace the TCP-based FramedSocket with a UDP-based equivalent that includes its own retransmission and ordering logic. They benchmark the throughput and latency of the UDP variant against the TCP variant under various loss conditions. The exercise illustrates the practical considerations behind transport selection in tunnel design and produces empirical data on the throughput-collapse pathology discussed in Chapter 3.

Exercise 9: A peer-to-peer mesh. Students refactor the architecture from server-as-switch to peer-as-switch, in which each connected client also acts as a switching plane for traffic destined to other clients. The exercise illustrates the architectural trade-offs between hub-and-spoke topology (Brooklyn Bridge's choice) and full mesh (the choice of products like Tailscale and ZeroTier).

17. Future Directions

This chapter identifies known limitations of the version 3.1.2 reference implementation and the directions in which subsequent versions are planned to develop. The intent is to be candid about what the bridge does not yet do and where its evolution is expected to take it.

17.1 Application-layer encryption (planned for v13)

The most operationally significant limitation of version 3.1.2 is the absence of application-layer encryption. The bridge relies on the underlay (typically ngrok's TLS) to provide confidentiality and authentication; it provides neither itself. This is acceptable for many of the bridge's current operational scenarios — small-shop deployments where the underlay is trusted — but it is unacceptable for operators whose threat model includes a malicious or compromised underlay.

A version 13 release is planned that adds TLS at the bridge's protocol layer. Each connection will perform a TLS handshake before the HELLO/CONFIG exchange; the server will present a certificate that the client verifies; the client will present a certificate that the server uses for both authentication and authorization (mapping the certificate's subject to a permitted client_id). The wire protocol above TLS will not change; the framing layer, message types, and per-session logic will operate identically over the encrypted channel.

17.2 Cross-platform clients (planned for v14)

Version 3.1.2 includes a Windows-only TAP wrapper. The protocol layer is platform-independent and operates correctly on Linux and macOS in the test harness, but no production Linux or macOS client exists. A version 14 release is planned that includes a Linux client using the /dev/net/tun interface and a macOS client using the Apple-supplied utun device.

The implementation effort is modest — approximately one hundred additional lines of platform-specific code per platform — but the testing effort is substantial. A multi-platform release requires CI infrastructure that runs the test harness on each target platform, integration testing that exercises real TAP/TUN devices on each platform, and operator documentation for each platform's particular deployment procedures. The release is therefore scheduled for after the version 13 encryption work, when the test harness will already have been audited as part of the encryption security review.

17.3 UDP underlay (planned for v15)

Version 3.1.2 uses TCP exclusively. As discussed in Chapters 3 and 5, the choice is operationally convenient and adequate for the bridge's current bandwidth profile, but it is not optimal for high-throughput scenarios. A version 15 release is planned that adds UDP support with optional reliable-delivery semantics implemented at the bridge layer.

The implementation will be more complex than the TCP variant because UDP does not provide ordering, reliability, or flow control. The bridge will need to implement, at minimum, a simple sequence-number-based reordering buffer and a NAK-based retransmission mechanism. Several existing protocols — KCP, QUIC, custom designs — could serve as templates; the choice will be made in light of the bridge's specific requirements during the version 15 design phase.

17.4 Asynchronous I/O for higher concurrency

The threading model used in version 3.1.2 imposes a per-client memory and scheduling cost that becomes significant at concurrencies above approximately fifty clients per server. A possible direction for future work is to rewrite the I/O paths in asyncio, which would permit a single server to host hundreds or thousands of clients on a single core. The complexity cost is non-trivial — asyncio code is notably harder to reason about than threaded code at this scale — and the operational benefit is uncertain because the bridge's design intent has emphasized small-scale deployments. Whether this direction is pursued depends on whether large-scale operators emerge as a priority audience.

17.5 Non-goals

Several directions that might be expected of a network product are explicit non-goals for Brooklyn Bridge. They are listed here so that operators and contributors are not surprised by their absence.

The bridge will not implement Layer-3 routing. It is, by design, a Layer-2 bridge. Operators requiring Layer-3 functionality should layer a Layer-3 product atop the bridge's flat broadcast domain or use a Layer-3 product directly.

The bridge will not implement complex policy enforcement (firewall rules, ACLs, deep packet inspection). Such facilities belong in dedicated security products, not in a switching plane. Operators requiring them should deploy them at the edges of the bridge — on the hosts attached to the bridge — rather than within the bridge itself.

The bridge will not implement a control plane for centralized management of multiple bridges. This kind of feature is the principal value proposition of commercial SD-WAN products; competing with them on this dimension would require effort disproportionate to the bridge's intended scope. Operators with multiple bridges should manage them through the operator's existing infrastructure-as-code tooling.

The bridge will not pursue formal protocol standardization. The wire protocol is documented in this book and Appendix A and is intended to be stable, but no IETF or other standards-body submission is planned.

18. Conclusion

Brooklyn Bridge is a small program. The smallness is its principal feature.

It is small enough that a single engineer can read the entire codebase in a weekend. It is small enough that an operator can audit every line that runs in production. It is small enough that a student can extend it as a course exercise without the modification getting lost in scaffolding. It is small enough that the design choices are visible and contestable; nothing is hidden in a framework, nothing is buried under abstraction layers, nothing requires the reader to trust that the underlying machinery is correct because the reader can see it directly.

The smallness is achieved through deliberate restriction of scope. The bridge does one thing: it carries Ethernet frames between connected hosts as if they were on a single switch. It does not encrypt; it does not route; it does not enforce policy; it does not negotiate parameters; it does not run on every platform; it does not scale to thousands of clients; it does not interoperate with commercial products' control planes. Each of these absences is, in commercial terms, a missing feature. In engineering terms, each is a choice that preserves the property of being small enough to read.

This choice is not for every operator. Operators whose use cases require any of the absent features should select a different product, and Chapter 4's matrix identifies the appropriate alternatives. Brooklyn Bridge is for operators whose use cases align with its choices: small-scale Layer-2 connectivity over a trusted underlay, where the operational complexity of a commercial SD-WAN product is unjustified, where the licensing terms of partially-closed mesh VPN products are unacceptable, and where the ability to read and modify the source is operationally valuable.

The book has documented Brooklyn Bridge in detail proportional to its size. The documentation is longer than the code — substantially so — because the operational and architectural context of the code is voluminous even when the code itself is concise. The reader who has reached this conclusion now has, in principle, the materials to deploy Brooklyn Bridge in production, to extend it for pedagogical use, to evaluate its position against competing products, and to contribute its evolution toward the planned future versions. Whether any individual reader will undertake any of these activities is, of course, beyond the book's reach. The book has supplied the materials; the use of the materials is the reader's.

The N2NHU Labs project continues. Brooklyn Bridge is one component of a larger collection of small, observable, GPL-licensed infrastructure tools. Subsequent books in the N2NHU series will document subsequent components in the same general manner: the operational rationale, the architectural overview, the position among competing products, the structural code walkthrough, the deployment and configuration guidance, and the future directions. Each book is intended to serve, like this one, the engineer who still reads source code and the student who will build what comes next.

Appendix A. Wire Protocol Specification

This appendix specifies the Brooklyn Bridge wire protocol in detail sufficient for an independent reimplementation. The specification reflects the protocol as implemented in version 3.1.2.

A.1 Underlay requirements

The protocol assumes a reliable, ordered, byte-oriented underlay between exactly one server and any number of clients. TCP satisfies all three requirements. Other transports satisfying the requirements (TLS over TCP, SSH-tunneled TCP, ngrok TCP, Cloudflare Tunnel TCP) are equivalent for protocol purposes.

A.2 Frame structure

Every protocol message consists of exactly two parts: a five-byte header and a variable-length payload. The header has the following layout:

Bytes 0-0: Message type. One unsigned octet.

Bytes 1-4: Payload length in octets. Unsigned 32-bit integer in network byte order (big-endian).

The payload consists of exactly the number of octets indicated by the length field, with content interpreted according to the message type. A length of zero is permitted and indicates an empty payload.

A.3 Message types

Code	Name	Description
`0x01`	`DATA_ETHERNET`	Encapsulated Ethernet frame. Payload is the raw Ethernet frame including header. Bidirectional.
`0x02`	`HELLO`	Client identification. Payload is UTF-8 JSON object with client_id, version, platform fields. Client to server only.
`0x03`	`CONFIG`	Server configuration response. Payload is UTF-8 JSON object with assigned_ip, subnet_mask, gateway, dns, mtu, session_key fields. Server to client only.
`0x04`	`HEARTBEAT`	Liveness signal. Empty payload. Bidirectional, though typically client to server.
`0x05`	`STATS`	Statistics report. Payload is UTF-8 JSON object. Reserved; not currently transmitted in v3.1.2.
`0x06`	`ERROR`	Fatal error. Payload is a UTF-8 human-readable error message. Server to client; followed by connection close.

A.4 Connection lifecycle

A connection proceeds through the following phases. (1) The client opens a TCP connection to the server's listening port. (2) The client sends a HELLO message. (3) The server allocates a session, performs IPAM, and

either sends a CONFIG message (success) or an ERROR message followed by close (failure). (4) The client, upon receiving CONFIG, applies the assigned configuration and begins exchanging DATA_ETHERNET messages with the server. (5) The client sends HEARTBEAT messages at the configured interval to maintain liveness. (6) When the client wishes to disconnect, it performs an orderly socket shutdown. (7) The server detects the disconnect, releases the session, and (after lease expiration) releases the IPAM allocation.

A.5 Size limits

Implementations MUST enforce an upper bound on the payload length. The reference implementation enforces 256 kilobytes (262144 octets) by default. Implementations MAY enforce a larger or smaller bound; the bound MUST be configured to a value that prevents resource exhaustion attacks but is large enough to accommodate the largest legitimate Ethernet frame plus protocol overhead. Implementations MUST drop messages whose declared payload length exceeds the configured bound and SHOULD close the connection upon detection of such a message.

A.6 Error handling

Implementations MUST handle the following error conditions gracefully: (a) a partial header read, in which the connection closes before all five header bytes are received; (b) a partial payload read, in which the connection closes before all declared payload bytes are received; (c) a payload length exceeding the configured bound; (d) a malformed JSON payload in HELLO, CONFIG, or STATS; (e) an unknown message type. The reference implementation handles each by either closing the connection (a, b, c, d for HELLO and CONFIG) or by counting and discarding (d for STATS, e). Alternative implementations MAY choose different responses but MUST NOT crash on any of these conditions.

Appendix B. Glossary

This appendix defines terms used throughout the book that may be unfamiliar to readers from outside networking or open-source software. Definitions are concise; readers requiring more detailed treatment of any term should consult standard networking texts.

Term	Definition
ARP	Address Resolution Protocol. The mechanism by which IP addresses are resolved to MAC addresses on a Layer-2 network. Operates by broadcast.
Broadcast domain	The set of hosts connected by Layer-2 links such that a broadcast frame from any host reaches all others. A flat Layer-2 network is a single broadcast domain; routers terminate broadcast domains.
CGNAT	Carrier-Grade Network Address Translation. A NAT operated by an Internet Service Provider in which many customer connections share a single public IP address. Customers cannot accept inbound connections through CGNAT.
Encapsulation	The operation of wrapping a frame or packet from one network in an envelope that is meaningful to another network, so that the wrapped object can traverse the second network. The reverse operation is decapsulation.
Ethernet frame	The unit of transmission on an Ethernet network. Consists of a destination MAC, source MAC, EtherType (or length), payload, and frame check sequence. Brooklyn Bridge tunnels Ethernet frames without their FCS.
`FramedSocket`	The Brooklyn Bridge class that implements length-prefixed framing over a TCP socket. Both server and client use FramedSocket as the protocol-layer entry point for the underlay.
IPAM	IP Address Management. The subsystem responsible for allocating IP addresses from a configured pool and tracking lease expirations.
MAC address	Media Access Control address. A 48-bit identifier of an Ethernet adapter. Locally administered MAC addresses (with the second-least-significant bit of the first octet set) are used in virtual environments such as Brooklyn Bridge.
MAC learning	The process by which an Ethernet switch (hardware or software) populates its forwarding table by observing the source MAC of frames as they arrive on each port.
ngrok	A commercial service that provides reverse tunnels from a public endpoint to a private host, traversing NAT. Brooklyn Bridge uses ngrok TCP tunnels as the default underlay.
Overlay network	A logical network constructed atop a different network. Brooklyn Bridge's overlay is a flat Ethernet broadcast domain; its underlay is a TCP connection.
Runt frame	An Ethernet frame shorter than the minimum legal size. Brooklyn Bridge defines this as anything less than 14 bytes (the size of an Ethernet header alone). Runts are dropped at both client and server ingress.
Session key	An ephemeral identifier generated by the Brooklyn Bridge server for each connection. Used as the key for IPAM allocations and SwitchPlane learn entries. Distinct from the operator-chosen client_id.
`SwitchPlane`	The Brooklyn Bridge class that implements the source-learning Ethernet switching algorithm. Owns the MAC forwarding table.
TAP device	A virtual Ethernet adapter implemented in software, which exposes a file-like interface for

Term	Definition
	transmitting and receiving frames. Distinct from a TUN device, which operates at Layer 3.
Token bucket	A rate-limiting algorithm in which tokens accumulate at a fixed rate up to a burst cap, and each operation consumes a token. Operations presented when the bucket is empty are denied.
Underlay	The carrying network of an overlay. Brooklyn Bridge's default underlay is an ngrok TCP tunnel.